

ParetoBandit: Autonomous Budget Pacing for Non-Stationary LLM Routing

Abstract

Large Language Model (LLM) routing dynamically directs user queries across a portfolio of models—from lightweight open weights to frontier APIs—optimizing the trade-off between inference cost and response quality. While static classifiers are commonly deployed for this task, production environments exhibit significant non-stationarity: models receive silent performance updates, inference providers drop prices abruptly, and aggregate API limits impose strict dollar budgets. In this paper, we present an open-source framework for non-stationary LLM routing via cost-aware contextual bandits. We demonstrate three key capabilities critical for production deployments: (1) an online Primal-Dual Lagrangian relaxation that enforces a strict per-request average-cost target dynamically over an open-ended request stream—without requiring a known time horizon—eliminating the need for offline hyperparameter tuning of cost penalties; (2) a geometric forgetting mechanism that enables the router to adapt gracefully to model quality shifts without suffering catastrophic forgetting of its initial offline-trained priors; and (3) the application of a rigorous hyperparameter tuning methodology to LLM routing that jointly optimizes exploration, prior strength, and forgetting factors. While established in other domains, adapting this tuning specifically addresses the uniquely contending requirements of LLM routing, allowing the system to strike an optimal balance between stable stationary exploitation and rapid agility under distribution shifts.

Through empirical evaluation on an established dataset of 1,785 realistic prompts, spanning a portfolio with a 500× cost differential (Llama-3.1-8B to Gemini-2.5-Pro), we demonstrate our system’s robustness. Under simulated reward drift, our approach significantly outperforms both frozen static policies and naive infinite-memory bandits. Critically, we demonstrate that when cost drift (e.g., a mid-cycle price drop) and budget constraints interact, our adaptive budget pacer automatically exploits the freed capital to route towards higher-quality frontier models, delivering up to +6.7% quality lift while holding spend within 2% of the dollar target. Static penalty approaches, by contrast, deviate from the budget by up to 162%, translating to either wasted quality headroom or SLA-violating cost overruns. We further show that the bandit’s exploration mechanism enables *cold-start model onboarding*: a single API call adds a previously unseen model with no offline evaluation or warmup priors, and the router discovers its quality–cost niche from live traffic

within hundreds of requests while maintaining budget compliance—eliminating the days-to-weeks recalibration cycle required by static routers. We release the fully documented routing framework to the open-source community to facilitate both further research and immediate production use.

ACM Reference Format:

. 2026. ParetoBandit: Autonomous Budget Pacing for Non-Stationary LLM Routing. In . ACM, New York, NY, USA, 32 pages. <https://doi.org/10.1145/1122445.1122456>

1 Introduction

Large language model (LLM) inference increasingly draws from multi-provider portfolios [6], spanning lightweight open-weights models to frontier APIs. Because the optimal model varies across tasks, domains, and individual prompts, **LLM routing**—dynamically selecting the best model per request—has emerged as a critical lever for managing cost and quality at scale [8, 13, 28, 31]. In production, an engineering team may route millions of requests daily across models spanning a 500× price range (e.g., \$0.00003/req for Llama-3.1-8B vs. \$0.015/req for Gemini-2.5-Pro) and must keep average spend within a strict API budget.

Crucially, real enterprise portfolios are not binary “strong-vs.-weak” pairs. Organizations typically maintain three or more model tiers—budget, mid-range, and frontier—and often deploy multiple models per tier for redundancy, latency, or specialization (e.g., a code-focused model alongside a general-purpose model at the same price point). The number of viable options continues to grow: major providers now offer 5–15 distinct model endpoints each, and multi-provider deployments routinely span $K=5$ –10 or more candidates. The value of a larger portfolio is not that more models necessarily produce a better quality frontier—a single frontier API may dominate on raw quality—but rather that more models at distinct price points give the router *more degrees of freedom to find quality within a budget*. When budgets are tight, a $K=2$ router can only choose “cheap” or “expensive”; with $K \geq 3$ it can discover that a mid-tier model matches frontier quality on easy prompts at a fraction of the cost, stretching the budget to afford frontier calls where they matter most. The same logic applies to non-stationarity: when a model degrades, a richer portfolio increases the chance that an alternative maintains quality at a comparable cost; when a provider cuts prices, more models mean more opportunities for the router to reallocate freed budget toward quality improvements.

Many existing routers operate as static policies trained offline on evaluation data, employing diverse architectures from matrix factorization [28] and graph neural networks [13] to contrastive representation learning [8] and cascading heuristics [6]. While highly effective in stationary environments, frozen policies risk degradation when deployed in real-world scenarios due to three critical factors. First, **deployment distributions drift**: prompt mixes vary by use case and drift over time from offline training data [3]. Second, **model quality is non-stationary**: providers

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference’17, Washington, DC, USA

© 2026 ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/1122445.1122456>

silently update APIs in ways that can subtly shift downstream performance on specific tasks [7, 24], meaning a router trained on historical data inherits a *stale policy*. Third, **model costs drop frequently**: rapid frontier releases often trigger price cuts [38]. If a static router maintains a fixed routing distribution during a price drop, it merely pockets the savings, leaving potential quality gains—such as upgrading a larger percentage of traffic to the now-cheaper frontier model—unexploited.

To address the need for online adaptation, recent work has formulated routing as a contextual bandit or reinforcement learning problem [22, 31, 34, 40]. These approaches learn dynamically from feedback (e.g., user ratings or automated judge scores) to adjust their policies over time, utilizing preference-conditioned bandits [22, 40] or acting as complex orchestration agents [34]. However, three critical challenges remain for deploying these adaptive systems in budget-constrained enterprise environments: 1. **Budget Enforcement**: Existing cost-aware systems typically either scalarize cost into the reward via a static penalty weight λ [28, 40] or rely on finite-horizon knapsack algorithms [31]. Static weights require manual offline tuning to translate an abstract penalty into a dollar outcome whenever pricing changes, while knapsack approaches require knowing the total number of queries in advance, making them unsuitable for continuous, open-ended production streams. 2. **Catastrophic Memory vs. Agility**: Standard algorithms like LinUCB often use infinite memory (forgetting factor $\gamma = 1.0$). While optimal for stationary settings, this causes the bandit to react far too slowly when a previously high-performing model suddenly degrades in quality, accumulating massive regret during the transition. 3. **Contending Contribution Requirements**: Tuning an adaptive system to simultaneously maintain stable stationary performance (which favors infinite memory and low exploration) and agile non-stationary adaptation (which favors aggressive forgetting and high exploration) creates conflicting optimization objectives. Standard cross-validation optimizes solely for stationary metrics, leading to brittle real-world deployments.

Left unaddressed, these gaps carry direct financial consequences: a static penalty that drifts from its budget target either wastes quality headroom (under-spend) or triggers cost overruns (over-spend), and a router that cannot detect a price drop leaves freed budget idle rather than reinvesting it in application quality.

We address these practical deployment challenges with **ParetoBandit**, an open-source contextual bandit framework explicitly designed for budget-constrained, non-stationary LLM routing. We make four key contributions, evaluated on a dataset of 1,785 prompts scored by a continuous three-factor rubric (cross-judge robustness validated in Appendix ??) across a canonical three-tier model portfolio:

- (1) **Direct Budget Pacing via Primal-Dual Lagrangian Relaxation**: We implement an online Lagrangian budget pacer (BudgetPacer) that accepts a direct dollar-denominated rate target (e.g., “\$0.005 per request”) and enforces it over an open-ended request stream with no known horizon. Unlike finite-horizon knapsack approaches [31] that require knowing the total number of requests in advance, our rate-constraint formulation—adapted from the Primal-Dual mechanism of Agrawal and Devanur [2]—self-regulates regardless of traffic volume. By applying

exponential moving average (EMA) smoothing to handle the extreme variance in LLM pricing, the system dynamically adjusts penalties to achieve exact budget compliance while automatically tracing the Pareto frontier, eliminating offline λ tuning.

- (2) **Robust Adaptation via Geometric Forgetting**: To combat silent model updates (reward drift), we integrate a geometric forgetting mechanism ($\gamma = 0.995$) coupled with offline-to-online warmup priors. This enables the router to adapt rapidly to quality shifts without suffering catastrophic forgetting of the initial prompt-preference mapping, drastically reducing cumulative regret under drift compared to naive bandits ($\gamma = 1.0$) and frozen policies.
- (3) **Exploiting Cost Drift under Budget Constraints**: We demonstrate that when a model provider drops its price mid-cycle, the BudgetPacer automatically detects and exploits the opportunity. Unlike static baselines that fail to adapt, the primal-dual mechanism automatically identifies the degree of constraint slack after the price change: λ_t decays as post-drop costs fall below the budget target, with the speed depending on the margin between realized costs and the target. ParetoBandit maintains precise Phase 1 budget compliance (0.98–1.00× target) and correctly relaxes when the constraint becomes slack, while delivering significant reward lifts (+0.017 to +0.054).
- (4) **Rigorous Hyperparameter Optimization for Shift**: We adapt joint hyperparameter tuning (exploration α , prior strength n_{eff} , forgetting factor γ) to the LLM routing domain. While utilized in other fields, this approach directly resolves the uniquely contending requirements of our system, structurally balancing stable stationary exploitation with rapid agility under distribution shifts where standard cross-validation fails.

Beyond these four contributions, we demonstrate that ParetoBandit’s exploration mechanism enables *cold-start model onboarding*: a single `register_model()` call integrates a previously unseen model—with no offline evaluation or warmup priors—into a budget-constrained router, and the bandit discovers its quality–cost niche from live traffic within hundreds of requests (Section 6.5; Appendix A.5).

By releasing the framework open-source, we aim to provide an immediate pathway for practitioners managing enterprise LLM gateways to replace manual recalibration with autonomous Primal-Dual pacing, and to offer the research community a reproducible testbed for non-stationary routing. The remainder of this paper is organized as follows: Section 2 surveys related work; Section 3 formulates the budget-constrained non-stationary routing problem; Section 4 presents the system architecture; Section 5 details the experimental setup; Section 6 discusses the empirical evaluation; and Section 7 outlines practical deployment considerations.

2 Related Work

LLM routing has evolved rapidly from cascading heuristics to supervised classifiers, and more recently, to adaptive routing systems. In this section, we situate ParetoBandit within the landscape of these approaches, highlighting the gaps in addressing production non-stationarity and dynamic budget constraints.

2.1 Static and Supervised Routers

Early approaches to cost-aware LLM routing relied on static rules or cascading heuristics (e.g., FrugalGPT [6]), where a system queries models sequentially from cheapest to most expensive, stopping when a predicted quality threshold is met. Recent works have mathematically unified single-model routing and cascading into a single linear optimization problem to maximize quality under a strict budget constraint. While simple to implement, cascades introduce highly variable latency, as complex queries must await sequential failures before reaching a capable model.

More recent systems formulate routing as a supervised classification or representation learning task trained offline on preference or evaluation data. RouteLLM [28] uses matrix factorization on pairwise preference data to map queries to models via a cost threshold. GraphRouter [13] constructs a heterogeneous graph over tasks, queries, and LLMs, using edge prediction to estimate per-model quality and cost, allowing it to generalize to newly added LLMs. Contrastive approaches have also gained traction: CSCR (Cost-Spectrum Contrastive Routing) explicitly penalizes model price during training to push embeddings of cheaper, capable models closer to the prompt, reducing runtime routing to an efficient nearest-neighbor lookup. Similarly, RouterDC [8] learns joint embeddings of queries and LLMs via dual contrastive losses. Other architectures use cross-attention mechanisms to explicitly model prompt-LLM feature interactions and predict both response quality and generation cost simultaneously. For theoretical rigor, predictive routers like CARROT jointly forecast accuracy and exact inference cost, providing minimax rate optimality guarantees for worst-case efficiency.

While supervised routers achieve strong performance when deployed on stationary distributions that match their training data, they exhibit two critical flaws in production environments. First, they deploy *frozen policies* that cannot adapt to non-stationarity (e.g., silent model updates or shifting user prompts). Second, they enforce budget constraints via fixed, offline-tuned thresholds or cost penalties. If a model provider unexpectedly drops prices, these static systems fail to dynamically exploit the new cost-quality trade-off, either under-spending their budget or requiring a manual, offline retraining and recalibration cycle.

2.2 Adaptive and Bandit-Based Routers

To enable online adaptation, a recent wave of research has formulated LLM routing as a contextual bandit or reinforcement learning (RL) problem, allowing systems to learn continuously from deployment feedback.

PILOT [31] extends LinUCB [21] with *preference-prior informed* initialization, injecting LLM embeddings learned offline from human preference data to mitigate the cold-start problem—an instance of the warm-start contextual bandit framework formalised by Kveton et al. [18]. Recent work has also explored using LLMs themselves to generate priors that jump-start bandit learning [30]. ParetoBandit adopts the same offline-to-online warm-start strategy, initialising the LinUCB sufficient statistics from a labeled reward dataset (Section 5), but differs from PILOT in two respects: it uses a continuous multi-factor reward signal rather than pairwise preferences, and it exposes the prior strength (n_{eff}) as a tunable hyperparameter that

controls the explore–exploit balance at initialisation (Section 6.4). To manage costs, PILOT employs an online multi-choice knapsack algorithm to optimize cumulative cost over a known, finite horizon. While theoretically grounded, the knapsack approach requires knowing the total number of queries in advance, making it difficult to apply to open-ended, continuous production streams where request volume fluctuates. Furthermore, PILOT uses standard LinUCB with infinite memory and provides no mechanism for non-stationarity; when a model silently degrades or a provider drops prices, the router cannot adapt without manual intervention.

BaRP [40] trains a contextual bandit using entropy-regularized policy gradients, conditioning routing on a user preference vector to allow inference-time adjustments to the cost-quality trade-off. LLM Bandit [22] utilizes a multi-armed bandit with preference-conditioned routing, incorporating model identity vectors to generalize to unseen models. xRouter [34] takes an RL approach, fine-tuning a small language model to act as an orchestration agent that decides whether to answer locally or invoke external APIs based on task complexity. Bandit algorithms have also been applied to ensembling: CaMVo uses a LinUCB bandit to dynamically select a subset of models to vote on an answer, halting the routing once a confidence threshold is met to save budget. Finally, recent work on contextual queueing bandits (e.g., ACQB [?]) leverages implicit feedback from user retries to jointly optimize routing and scheduling in conversational services.

2.3 Positioning ParetoBandit

ParetoBandit builds upon the foundation of contextual bandits but introduces critical structural advances designed specifically for the realities of open-ended, budget-constrained production environments:

- (1) **Continuous Budget Pacing:** Unlike PILOT’s finite-horizon knapsack—which requires knowing the total number of queries T in advance—or the static scalarized cost penalties used in RouteLLM and BaRP, ParetoBandit enforces a per-request average cost rate target via an online Primal-Dual Lagrangian relaxation (adapting the dual-variable mechanism of Agrawal and Devanur [2] from a cumulative knapsack constraint to a rate constraint suitable for open-ended production streams). This *BudgetPacer* strictly enforces a per-request average cost target (e.g., “\$0.005 per request”) over an unbounded horizon, automatically adjusting to price drops or traffic shifts without offline recalibration.
- (2) **Explicit Handling of Non-Stationarity:** While existing bandit routers continuously learn, they often employ infinite memory (e.g., standard LinUCB). When model quality degrades (reward drift), infinite memory causes catastrophic inertia, as the router slowly unlearns thousands of stale historical observations. ParetoBandit introduces a principled geometric forgetting mechanism that dynamically discounts stale evidence, enabling rapid adaptation to quality shifts while preserving enough historical context to avoid volatile, uninformed routing.
- (3) **Production State Management:** Existing bandit routers maintain learned state exclusively in process memory, making them unable to recover from restarts or incorporate delayed human

feedback (RLHF). ParetoBandit provides pluggable context persistence (in-memory or SQLite-backed) for asynchronous feedback ingestion and atomic state checkpointing for crash recovery, bridging the gap between research prototypes and deployable systems (Section 7).

- (4) **Native $K \geq 3$ Multi-Model Support:** Most existing routers—both supervised (RouteLLM [28]) and adaptive (BaRP [40])—are designed around a binary strong/weak decision, using pairwise preference data or binary cost thresholds to triage between exactly two models. This $K=2$ framing limits a router’s ability to operate under budget constraints and non-stationarity. With only two arms, the router has a single lever—what fraction of traffic goes to the expensive model—leaving no room to discover that a mid-tier alternative delivers comparable quality at a fraction of the cost. When rewards shift or prices drop, a richer portfolio gives the router more options to reallocate traffic and maintain quality within budget; with $K=2$, every such event collapses to the same binary trade-off. ParetoBandit’s contextual bandit formulation is natively K -armed: each model is an independent arm with its own learned reward surface, the BudgetPacer’s Lagrangian penalty operates over the full cost spectrum, and new arms are added at runtime via a single API call without architectural changes (Appendix A.5).

3 Problem Formulation

We formalize LLM routing as an online decision-making problem under budget constraints, where the environment is non-stationary with respect to both model quality (reward drift) and model pricing (cost drift).

3.1 Contextual Bandit Routing

The key premise of contextual routing is that *no single model is universally best*—the optimal choice depends on the prompt. LLMs exhibit heterogeneous capabilities across prompt types: a budget model may handle straightforward factual look-ups as well as a frontier API, while only the frontier model produces acceptable responses on complex multi-step reasoning or nuanced creative writing. A context-free router that ignores prompt content must commit to a single, fixed traffic distribution (e.g., 70% budget, 25% mid-tier, 5% frontier) regardless of prompt difficulty. Under a budget constraint, this is wasteful: frontier calls are spent on easy prompts where a cheap model would suffice, while hard prompts that justify the 500× cost premium receive frontier service no more often than easy ones. Conditioning on the prompt allows the router to learn *which* prompt types each model excels at, reserving expensive calls for the prompts where they yield the largest quality gain and routing the rest to cheaper alternatives. This selective spending is what enables a contextual router to trace the quality–cost Pareto frontier—and it is impossible without a representation of the prompt that captures the task-level distinctions relevant to model selection.

Let $t \in \{1, 2, \dots, T\}$ denote a discrete sequence of user requests (prompts). At each step t , the router receives a context vector $x_t \in \mathbb{R}^d$ representing the prompt, derived from a pre-trained embedding model (e.g., SentenceTransformers). The router must select a single language model a_t from a discrete portfolio of available models $\mathcal{A} = \{1, \dots, K\}$.

Upon executing the prompt on model a_t , the system observes a scalar reward $r_t \in [0, 1]$ indicating the quality of the response. This reward may be derived from an automated LLM-as-a-judge, explicit user feedback (e.g., thumbs up/down), or an implicit task completion metric. Crucially, the router operates under *bandit feedback*: it only observes the reward for the chosen model a_t and does not know what the counterfactual rewards would have been for the unselected models.

The primary objective is to learn a routing policy π that maximizes the expected cumulative reward:

$$\max_{\pi} \mathbb{E} \left[\sum_{t=1}^T r_{t, \pi(x_t)} \right] \quad (1)$$

In words: the router aims to make the best model choice on every request so that, summed over all T requests, the total quality is as high as possible—even though it never observes what would have happened had it chosen differently.

3.2 The Budget Constraint

In production, each model $a \in \mathcal{A}$ incurs a known, deterministic inference cost $c_a(x_t)$ (typically priced per input and output token). Because the models in a portfolio often span orders of magnitude in price—from open-weights models costing fractions of a cent to frontier APIs costing several cents per request—the router must strictly manage aggregate spend.

We enforce this constraint via an online Primal-Dual Lagrangian relaxation, adapting the dual-variable mechanism of Agrawal and Devanur [2] from a finite-horizon cumulative knapsack to a *rate constraint* suited for open-ended deployment. The system is given a target average budget B (e.g., “average \$0.005 per req”). The policy π must maximize cumulative reward subject to the constraint that the long-run average cost does not exceed B :

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T c_{\pi(x_t)}(x_t) \leq B \quad (2)$$

In plain terms: averaged over many requests, the cost per request must stay at or below the target B . The router may occasionally splurge on an expensive frontier model, but must offset that with cheaper choices elsewhere so the running average never exceeds the budget.

Rate constraint vs. cumulative knapsack. The classical Contextual Bandit with Knapsacks (CBwK) formulation [2] assumes a cumulative budget $\sum_{t=1}^T c_t \leq B_{\text{total}}$ over a known, finite horizon T , with dual updates that account for the remaining budget fraction. Production LLM routing is fundamentally open-ended: request volume is unknown and variable, and the operator’s natural specification is a cost *rate* (dollars per request), not a finite pool. Our formulation inherits the Primal-Dual decomposition—splitting the constrained routing problem into unconstrained bandit subproblems coupled through a dual variable λ_t —but the dual update targets the running average cost rather than a depleting budget, making the system horizon-free: it self-regulates at any traffic volume without requiring T as input. The finite-horizon sublinear regret guarantees of the original CBwK analysis do not directly apply to this rate-constraint setting; we validate convergence and budget compliance empirically (Sections 6–7). This design choice carries a tradeoff. Because

the rate constraint tracks an EMA-smoothed average rather than a cumulative total, a sustained shift toward expensive requests can temporarily push realized spend above B until the dual variable corrects—a lag bounded by the EMA half-life (~ 14 observations at $\alpha_{\text{ema}}=0.05$). Conversely, a finite-horizon knapsack that underestimates T exhausts its budget early, forcing service degradation for the entire remaining tail of the horizon. We argue in Section 7 that the rate constraint’s bounded transient overshoot is strictly preferable to the knapsack’s catastrophic quality cliff under traffic variability.

The standard approach to this constraint in the routing literature is to scalarize the reward using a fixed penalty weight $\lambda > 0$, optimizing a penalized utility $u_{a,t} = r_{a,t} - \lambda c_a(x_t)$ [28]. However, selecting λ requires extensive offline simulation on historical data to find the weight that happens to yield an average cost of B . If the distribution of prompts shifts, or if the portfolio changes, the realized cost will drift away from B , requiring manual recalibration.

3.3 Non-Stationarity

Real-world LLM deployments are fundamentally non-stationary. We define two specific types of environment drift that a production router must handle autonomously:

Reward Drift (Quality Shifts). LLM providers silently update their models behind APIs, or internal teams push new versions of fine-tuned open-weight models. Formally, the conditional expected reward function $f_a(x) = \mathbb{E}[r_t | x, a]$ changes at some unknown time τ , such that $f_a^{(t < \tau)}(x) \neq f_a^{(t \geq \tau)}(x)$. Put simply, the expected quality of a model’s response to a given prompt is one value before the update and a different value after it—and the router does not know when the change occurred. A router must detect this shift and rapidly unlearn its historical estimates for model a , avoiding a stale policy that continues routing traffic to a degraded model.

Cost Drift (Price Drops). API providers frequently reduce inference prices to remain competitive [38]. Formally, the cost function $c_a(x)$ decreases at time τ . If the router uses a fixed, offline-tuned penalty λ , it will continue to apply the same relative penalty to model a . While this mechanically reduces the total spend (pocketing the savings), it fails to recognize that model a is now a relatively better value. An optimal router should dynamically exploit the newly freed budget to route more challenging queries to the newly-cheaper, higher-quality model, maximizing overall application quality while remaining precisely at the budget limit B .

4 Methodology: The ParetoBandit System

We present the architecture of ParetoBandit, an open-source contextual bandit routing framework. At its core, LLM routing is an *exploration–exploitation problem*: the router must balance selecting models it already knows to be good (exploitation) against trying uncertain or newly added models that might be better (exploration). Maximising long-term expected reward requires getting this trade-off right—and in production the tradeoff is complicated by two additional forces: *budget constraints* that limit how much exploration the router can afford, and *non-stationarity* that forces continuous re-exploration as model quality and pricing change.

ParetoBandit exposes a small set of interpretable hyperparameters, each governing a distinct facet of this tradeoff:

- α (exploration rate): how aggressively the router tries uncertain models versus exploiting known-good ones (Section 4.2).
- n_{eff} (prior strength): how strongly the router trusts its offline priors versus new online evidence—controlling how quickly exploration begins (Section 5).
- γ (forgetting factor): how quickly the router discounts stale evidence, determining when it *re-explores* arms whose quality may have changed (Section 4.3).
- B (budget target): a dollar constraint that bounds the cost of exploration, forcing the router to explore efficiently within its financial envelope (Section 4.2).

Every experiment in this paper examines how these knobs interact under the realities of budget-constrained, non-stationary deployment: stationary budget pacing (Section 6.1), cost drift (Section 6.2), catastrophic model failure (Section 6.3), joint hyperparameter selection (Section 6.4), and cold-start model onboarding (Section 6.5).

Design rationale. No single existing algorithm simultaneously learns reward surfaces from bandit feedback, enforces a long-run budget constraint, and adapts to non-stationarity. ParetoBandit’s architecture is therefore an evolution that composes four foundational results, each addressing one facet of the problem. The architectural cornerstone is the Primal-Dual Lagrangian relaxation of Agrawal and Devanur [2], whose decomposition of a constrained bandit into unconstrained subproblems coupled through a dual variable λ_t that encodes budget pressure. This composability is what makes the design modular: the inner reward learner and the outer budget mechanism evolve independently, and the choice of inner algorithm is a free design parameter.

For the inner learner we adopt LinUCB [21], which models expected reward as a linear function of the context and maintains closed-form sufficient statistics (A_a, b_a) per arm. Standard LinUCB, however, implicitly assumes two conditions that fail in production LLM routing: *stationarity* (reward distributions do not change) and *prior correctness* (any initialisation is either uninformative or accurate). In practice, providers silently update models, pricing shifts, and prompt distributions drift—the canonical concept drift problem [7, 14]—invalidating both assumptions. Offline warmup priors, trained on historical data, may already be partially stale by the time the router is deployed. We therefore extend LinUCB with two robustness mechanisms, a *forgetting factor* γ and a *tunable prior strength* n_{eff} , that address each failure mode. Crucially, LinUCB’s sufficient-statistic representation is what makes these extensions lightweight—the principal alternatives (Thompson Sampling, neural bandits, policy-gradient methods [40]) do not jointly support them without substantial algorithmic redesign:

- (1) *Non-stationary adaptation via γ .* Geometric forgetting reduces to a scalar multiply ($A_a \leftarrow \gamma A_a, b_a \leftarrow \gamma b_a$), directly realising the discounted-UCB principle of Garivier and Moulines [16] without redesigning the update rule. The parameter $\gamma \in (0, 1]$ controls the effective memory horizon: recent evidence dominates, allowing the router to track reward drift, while historical observations are geometrically discounted rather than erased.
- (2) *Prior-strength control via n_{eff} .* Warm-start initialization is a single matrix addition to (A_a, b_a) , instantiating the framework

of Kveton et al. [18]. The tunable prior strength n_{eff} acts as a safety valve against prior–reality mismatch: it sets how many pseudo-observations the offline priors contribute, controlling how quickly online evidence can override potentially stale offline estimates. A conservative n_{eff} lets the router begin learning from live traffic almost immediately; a large n_{eff} trusts priors longer, trading adaptability for early stability. Appendix A.4 quantifies this trade-off: directionally correct priors reduce regret at every n_{eff} , but systematically inverted priors cause harm that scales with n_{eff} , making the choice of prior strength a critical deployment decision.

- (3) *Computational efficiency* follows from Sherman–Morrison rank-1 updates at $O(d^2)$ per step, keeping the router on the critical path at $< 40 \mu\text{s}$ per decision (Appendix A.1).

While each ingredient is well-established individually, their composition introduces interactions that no single foundational paper analyses. Geometric forgetting shrinks A_a toward singularity over time, which inflates LinUCB’s confidence bonus, which increases the probability of selecting expensive arms, which in turn drives up λ_t —a feedback loop between the forgetting mechanism and the budget pacer that must be jointly calibrated. The EMA smoothing in the dual update (Eq. 5) addresses a further practical challenge—the $500\times$ cost variance across LLM tiers—that the original finite-horizon analysis does not consider. The hyperparameter sweep in Section 6.4 exists precisely to resolve these cross-component interactions under the contending demands of stability and agility.

4.1 System Architecture Overview

ParetoBandit operates as an asynchronous, closed-loop routing system (Figure 1). At inference time (step t), a user query x_t is processed by a lightweight feature extractor (e.g., a SentenceTransformer embedding model), which passes the query vector to the BanditRouter. The router maintains a set of sufficient statistics for each model $a \in \mathcal{A}$, optionally initialized via *offline-to-online warmup priors* that provide informed routing from the first request, eliminating the cold-start transient that would otherwise occur during initial deployment. The router computes a score for each model, incorporating uncertainty (LinUCB) and subtracting a dynamic cost penalty provided by the BudgetPacer. The router then selects the model a_t with the highest score, returning its response to the user.

After the query completes and is evaluated (either synchronously by an automated judge, or asynchronously via delayed user feedback), the reward r_t and exact cost c_t are fed back into the system. The BanditRouter updates its reward estimates (incorporating geometric forgetting), while the BudgetPacer updates its internal dual variable λ_t , adjusting the cost penalties for future requests to steer the system back toward the target budget B .

4.2 Primal-Dual Budget Pacing

To enforce strict, dynamically adjusting budget constraints without manual offline tuning of a static penalty weight λ_s , ParetoBandit employs the BudgetPacer, an online Primal-Dual Lagrangian relaxation inspired by Agrawal and Devanur [2] but adapted from a cumulative knapsack constraint to a per-request rate constraint suitable for open-ended production streams (Section 3).

At each step t , the router selects model a_t by maximizing a budget-augmented utility function:

$$a_t = \arg \max_{a \in \mathcal{A}_t} \left[\underbrace{\hat{\theta}_a^\top x_t + \alpha \sqrt{x_t^\top A_a^{-1} x_t}}_{\text{LinUCB score [21]}} - \underbrace{\lambda_s \tilde{c}_a}_{\text{static penalty}} - \underbrace{\lambda_t \tilde{c}_a}_{\text{dual penalty}} \right] \quad (3)$$

The first term, $\hat{\theta}_a^\top x_t$, is the router’s current best estimate of the expected reward for model a on prompt x_t (*exploitation*). The second term, $\alpha \sqrt{x_t^\top A_a^{-1} x_t}$, is a confidence bonus proportional to the router’s *uncertainty* about that estimate: models that have been selected rarely, or on dissimilar prompts, have larger bonuses, encouraging the router to try them (*exploration*). The exploration rate $\alpha > 0$ directly controls this tradeoff: higher values widen the confidence interval, driving the router to explore uncertain arms more aggressively at the cost of short-term reward; lower values concentrate traffic on the current best arm. The cost penalty terms ($\lambda_s \tilde{c}_a + \lambda_t \tilde{c}_a$) reduce the effective score of expensive models, constraining exploration to arms the budget can sustain—a mechanism that becomes critical when an uninformed, newly onboarded model also happens to be expensive (Section 6.5).

Here $\mathcal{A}_t \subseteq \mathcal{A}$ is the set of models surviving a hard cost ceiling (detailed below), λ_t is a Lagrange multiplier updated after observing the realized cost c_t , and $\tilde{c}_a \in [0, 1]$ is the log-normalized unit cost defined as

$$\tilde{c}_a = \frac{\log c_a - \log c_{\min}}{\log c_{\max} - \log c_{\min}}, \quad (4)$$

with $c_{\min}$ and $c_{\max}$ fixed to the market-floor and market-ceiling blended rates of the model portfolio, respectively. Values at or below $c_{\min}$ map to 0; values at or above $c_{\max}$ map to 1. The logarithmic scale compresses the $500\times$ cost differential into a bounded penalty that is commensurate with the $[0, 1]$ reward scale. Intuitively, the log captures the fact that jumping from a \$0.00003 model to a \$0.003 model ($100\times$) is a comparable “tier leap” to jumping from \$0.003 to \$0.015 ($5\times$); on a linear scale the first gap would be invisible while the second would dominate the penalty, distorting the router’s cost–quality reasoning.

Dual-Variable Dynamics and EMA Smoothing. The core innovation of the BudgetPacer is how it tracks and updates the dual variable λ_t in response to the extreme variance in LLM pricing. The mechanism works like a cruise-control system: when recent spending runs above the budget target, λ_t increases, making expensive models less attractive and pushing the router toward cheaper options; when spending dips below target, λ_t decreases, releasing the router to choose higher-quality (costlier) alternatives. Rather than updating λ_t directly from the raw per-request cost c_t , the pacer computes an *EMA-smoothed* cost signal:

$$\tilde{c}_t = (1 - \alpha_{\text{ema}}) \tilde{c}_{t-1} + \alpha_{\text{ema}} c_t \quad (5)$$

$$\lambda_{t+1} = \min\left(\bar{\lambda}, \max(0, \lambda_t + \eta (\tilde{c}_t/B - 1))\right) \quad (6)$$

where B is the user’s desired average cost per request, $\alpha_{\text{ema}} = 0.05$ is the smoothing factor, $\eta = 0.05$ is the step size, and $\bar{\lambda} = 5$ is a capacity cap.

Using the EMA $\tilde{c}_t$ is critical. If a single \$0.015 Gemini request were routed, the raw ratio c_t/B could spike to $500\times$ the target B (e.g., if $B = \$0.00003$). This would produce massive, sawtooth oscillations

Synchronous Inference Path

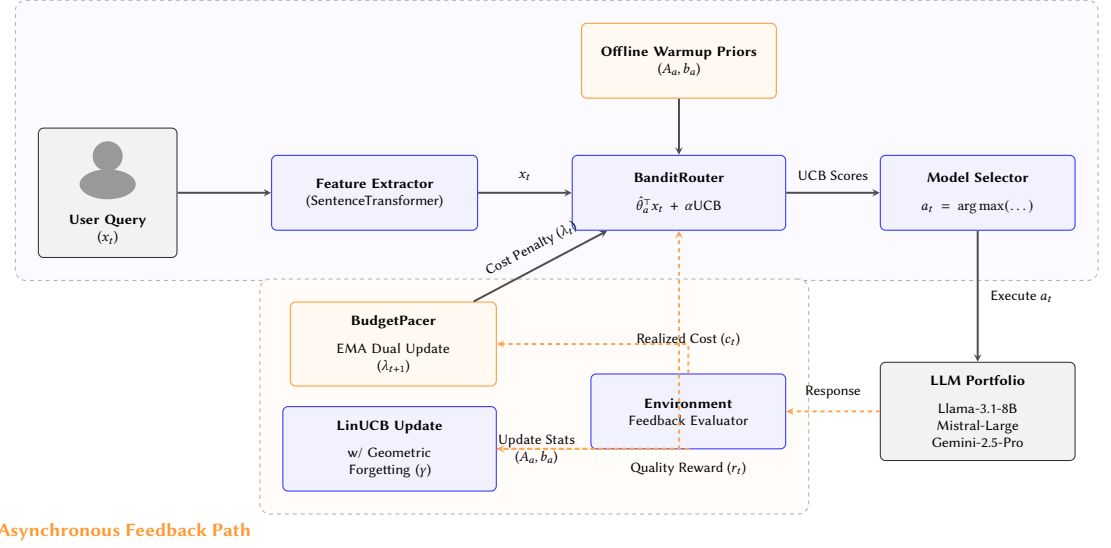


Figure 1: ParetoBandit System Architecture. The system cleanly separates synchronous inference (blue) from asynchronous feedback (orange, color-blind friendly). During inference, the BudgetPacer dynamically applies a dual cost penalty (λ_t) to the LinUCB scores to enforce the budget constraint B . As feedback arrives asynchronously, the reward r_t updates the router’s statistics via geometric forgetting (γ), while the exact cost c_t updates the pacer’s dual variable (λ_{t+1}) via an EMA smoother.

in λ_t that never converge, rendering the penalty useless. The EMA smooths this variance, generating a stable gradient signal whose magnitude reflects the *average* cost regime over recent history.

Furthermore, normalizing the gradient by B (i.e., $\tilde{c}_t/B$) makes the step size η portfolio-independent. The same η produces equivalent λ_t trajectories whether the cheapest model costs \$0.00003 or \$0.03, eliminating the need to retune hyperparameters when the pricing scale changes.

Two-Layer Enforcement (Adaptive Mode). The BudgetPacer combines two enforcement mechanisms simultaneously:

- **Hard ceiling (Safety Net):** When $\lambda_t > 0$, the candidate set $\mathcal{A}_t$ actively excludes models whose blended unit price exceeds the dynamic ceiling. This acts as a circuit breaker, preventing catastrophic overspend on any single request when the budget is severely constrained.
- **Soft penalty (Optimizer):** The dual penalty $\lambda_t \tilde{c}_a$ biases the UCB score toward cheaper models proportionally to their cost. This allows fine-grained, context-dependent routing within the surviving candidate set, minimizing quality loss within the hard budget envelope.

4.3 Non-Stationary Adaptation via Geometric Forgetting

LLM routing in production is subject to *concept drift* [14]: the joint distribution of prompts, model quality, and pricing shifts over time as providers silently update models [7, 10], user workloads evolve, and API pricing changes. A standard LinUCB algorithm ($\gamma = 1.0$) accumulates infinite memory, severely slowing adaptation to a shifting reward landscape because it must overcome thousands of

historical observations before its estimates reflect the new regime. Conversely, entirely discarding history discards the valuable prior knowledge required for stable, informed routing.

ParetoBandit addresses non-stationarity through a *forgetting factor* $\gamma \in (0, 1]$ applied directly to the disjoint LinUCB sufficient statistics. At each step t , before incorporating the new observation (x_t, a_t, r_t) , the design matrix and reward accumulator for the chosen arm a_t are exponentially discounted:

$$A_{a_t} \leftarrow \gamma A_{a_t} + x_t x_t^\top \quad (7)$$

$$b_{a_t} \leftarrow \gamma b_{a_t} + r_t x_t \quad (8)$$

This exponential weighting [16] gives observations an effective half-life of $\ln 2/(1 - \gamma)$ steps (≈ 139 steps for $\gamma=0.995$) and an e-folding time of $1/(1 - \gamma)=200$ steps. Concretely, an observation from 200 steps ago retains weight $\gamma^{200} \approx e^{-1} \approx 0.37$. If a model that was previously excellent suddenly degrades, the router notices within a few hundred requests rather than remaining anchored by thousands of stale observations. We empirically find that $\gamma = 0.995$ strikes an optimal balance, creating an effective memory window of approximately 200 steps (see Section 6.4 for the formal selection procedure). Recent evidence dominates, allowing the router to adapt to shifts rapidly, while older observations—including the offline warmup priors—are geometrically discounted. After $\sim 1,000$ steps, the prior’s contribution is reduced by $\gamma^{1000} \approx 0.007$; after a full deployment ramp-up ($\sim 1,800$ steps), priors are effectively replaced by online evidence. Warmup priors thus serve as a *deployment latency optimisation*: they provide informed routing from request #1, eliminating the cold-start cost that would otherwise accumulate

during the initial learning phase, but the router's steady-state quality is determined by online evidence regardless of initialisation (Appendix A.3).

The parameter γ thus controls a bias–variance tradeoff between *stability* (trusting historical evidence to maintain high baseline quality) and *plasticity* (responding quickly to new evidence when models degrade).

4.4 Computational Efficiency

Because the router sits on the critical path of every LLM request, per-step latency must be negligible relative to model inference time. We ensure this through three design choices.

$O(d^2)$ updates via Sherman-Morrison. Naive LinUCB recomputes A_a^{-1} from scratch after each observation, costing $O(d^3)$. ParetoBandit instead maintains a cached inverse A_a^{-1} and applies the Sherman-Morrison rank-1 update formula:

$$A_a^{-1} \leftarrow A_a^{-1} - \frac{(A_a^{-1}x_t)(x_t^\top A_a^{-1})}{1 + x_t^\top A_a^{-1}x_t}, \quad (9)$$

which incrementally adjusts the existing inverse to account for the new data point x_t , rather than recomputing the entire inverse from scratch—reducing each update to $O(d^2)$. For the production feature dimension $d=26$, this is $\sim 17,500\times$ faster than a full $O(d^3)$ inversion. A full $O(d^3)$ recomputation is triggered only as a rare numerical-stability safeguard when the regularization floor decays below 10% of its initial value under prolonged forgetting—in practice fewer than one in every thousand updates.

$O(Kd)$ arm selection. The expected reward $\hat{\theta}_a^\top x_t$ is a single $O(d)$ dot product per arm because $\hat{\theta}_a = A_a^{-1}b_a$ is cached and refreshed only at update time. The UCB exploration bonus requires one $O(d^2)$ quadratic form $x_t^\top A_a^{-1}x_t$ per arm. With $K=3$ arms and $d=26$, the full selection pass amounts to $\sim 2,100$ floating-point operations—well under a microsecond on commodity hardware.

Constant-time geometric decay. When $\gamma < 1$, discounting $A_a \leftarrow \gamma A_a$ requires multiplying the cached inverse by the reciprocal: $A_a^{-1} \leftarrow A_a^{-1}/\gamma$ —a scalar division that adds $O(d^2)$ element-wise operations but avoids a full matrix rebuild. Combined with the Sherman-Morrison correction for the new observation, the total per-update cost remains $O(d^2)$ regardless of whether forgetting is active.

Empirical validation. A latency microbenchmark (Appendix A.1) confirms that the production configuration ($d=26$, Sherman-Morrison) completes a full route + update cycle in $39\mu\text{s}$ (p50) and sustains $> 24,000$ req/s on a single CPU core. The combined effect of PCA dimensionality reduction and incremental inverse caching yields a $120\times$ throughput improvement over a naive full-inversion baseline operating on raw 384-D embeddings. Including prompt embedding and PCA projection, the full end-to-end routing pipeline completes in ~ 8 ms on CPU—adding $< 1\%$ overhead even to the fastest production LLM call (Claude Haiku 4.5 at 952 ms total latency [15]) and $< 0.2\%$ for typical workloads.

Despite the centrality of latency to production routing, no peer-reviewed LLM router—including bandit-based systems (LLM Bandit [22], BaRP [40], MixLLM [41]), classifier-based routers (RouteLLM [28],

CSCR [36]), or RL-based routers (PROTEUS [5])—reports per-decision or end-to-end routing latency. An end-to-end decomposition of ParetoBandit's critical path (Appendix A.1, Table 7) reveals that the routing decision contributes $< 1\%$ of total overhead ($22\mu\text{s}$ out of ~ 8 ms): prompt embedding dominates at 97%, a cost shared by every contextual router. The full ~ 8 ms pipeline runs entirely on CPU and is competitive with the fastest reported LLM routing systems, including GPU-accelerated neural classifiers [23], without requiring a dedicated accelerator for the routing decision.

5 Experimental Setup

We evaluate ParetoBandit on a canonical portfolio of three LLMs, spanning a large cost differential, and using realistic prompt data to simulate both stationary and non-stationary production environments. Table 1 summarises the data splits and model portfolio.

5.1 Datasets and Ground Truth

We collect 11,983 prompts from the LMSYS Chatbot Arena [44], reflecting real-world, open-ended user interactions spanning coding, reasoning, and conversational tasks. Each prompt is routed to all three models in the portfolio and evaluated by an automated LLM-as-a-judge.

Quality evaluation. We employ DeepSeek-R1 [11] as the evaluator via a structured chain-of-thought (CoT) judge prompt. The prompt first elicits a concise reasoning analysis of the response, then requests scores on three continuous dimensions ($[0, 1]$): *Reasoning Quality*, *Instruction Following*, and *Communication Quality*. The composite scalar reward is the weighted average $r_t = 0.4 \cdot s_{\text{reason}} + 0.3 \cdot s_{\text{instr}} + 0.3 \cdot s_{\text{comm}} \in [0, 1]$. This continuous rubric provides the granular quality distinctions necessary for multi-arm routing; in the LLM routing setting, binary win/loss outcomes collapse into ties on a large fraction of prompts because adjacent-tier models produce comparable quality on most queries, rendering binary signals insufficient for distinguishing among $K > 2$ models (Section 7 provides a detailed analysis and supporting evidence from the literature). The full judge prompt is reproduced in Appendix ??.

For online learning, the bandit's convergence rate scales directly with the reward gap between the optimal and suboptimal arms [19]; we therefore select the judge that maximises routing margins. DeepSeek-R1 produces the largest inter-model gaps and the highest fraction of actionable prompts among the judges we evaluated, while its oracle routing decisions capture $\geq 97\%$ of two independent supplementary judges' oracle reward (Appendix ??). Recent multi-judge aggregation methods [39, 43] target confounder-induced bias rather than the signal compression inherent in averaging judges with different scale ranges; applied to our panel, averaging reduces routing margins by $\sim 30\%$ (Appendix, Figure ??), making the single maximally discriminative judge the principled choice for bandit learning.

Data splits. The 11,983 prompts are partitioned into three disjoint sets via stratified sampling (by oracle difficulty, seed=42, 70/15/15 ratio). No prompt appears in more than one split:

- (1) **Train** ($n=8,374$): Used *offline* to build warmup priors that initialise the bandit's sufficient statistics (A_a, b_a), addressing the cold-start problem. Never seen during online evaluation.

Table 1: Dataset splits and model portfolio ($K=3$). All prompts are sourced from LMSYS Chatbot Arena and judged by DeepSeek-R1. The 500 $\times$ cost differential between the cheapest and most expensive model stresses the router’s quality–cost tradeoff.

Split	n	Role
Train	8,374	Warmup priors (offline)
Val	1,785	Online learning, hparam tuning
Test	1,824	Online evaluation (held-out data)
Total	11,983	

Model	Tier	Cost (\$/req)
Llama-3.1-8B	Budget	\$0.00003
Mistral-Large	Mid-tier	\$0.003
Gemini-2.5-Pro	Frontier	\$0.015

- (2) **Val** ($n=1,785$): Used for online learning and hyperparameter selection (Section 6.4) under stationary conditions.
- (3) **Test** ($n=1,824$): Evaluation split, held out from both warmup-prior fitting and hyperparameter selection. The bandit policy continues to receive feedback and adapt during the test trajectory, consistent with the standard cumulative-regret protocol for online learning [19]; “held-out” refers to data isolation, not policy freezing. Non-stationary scenarios (reward drift, cost drift) are constructed by manipulating reward or cost columns at a mid-trajectory phase boundary. All reported results use this split, averaged over multiple random seeds. The val burn-in is a production-realistic protocol that adds no statistically significant advantage to test-set performance: warmup priors alone (0% burn-in) achieve equivalent regret (74.0 vs. 74.4, $p=0.64$, 20 seeds), and this finding holds under budget-paced conditions (tight: $p=0.40$; moderate: $p=0.74$); the burn-in’s purpose is calibration under deployment conditions, not artificial performance inflation (Appendix A.3).

Context representation. Contextual routing rests on the inductive bias that *semantically similar prompts benefit from the same model*: a coding question, a creative-writing task, and a factual look-up each exercise different model capabilities, so a representation that captures these semantic axes gives the bandit a signal for differential routing. Dense sentence embeddings are the consensus feature choice across the LLM routing literature—adopted by supervised systems (RouteLLM [28], RouterDC [8], GraphRouter [13]), bandit-based routers (PILOT [31], MixLLM [41]), and RL approaches (PROTEUS [5])—because they compress a variable-length prompt into a fixed-dimensional vector that preserves the task-level distinctions relevant to model selection.

Each prompt is encoded by the all-MiniLM-L6-v2 Sentence-Transformer [35] (384-D), L2-normalized, and projected to a 25-dimensional context vector via PCA, followed by a bias term ($d=26$). Dimensionality reduction is essential because LinUCB maintains a $d \times d$ design matrix A_a per arm, making per-step updates $O(d^2)$; projecting from 384 to 25 dimensions reduces this cost by $\sim 235\times$. We choose PCA over nonlinear alternatives (e.g., autoencoders) because LinUCB assumes a linear reward model $\mathbb{E}[r \mid x] = \theta^\top x$;

PCA is the variance-maximizing linear projection, preserving the most signal under the same linearity assumption the bandit already makes. Although 25 components retain only $\sim 29\%$ of total embedding variance, the routing-relevant signal—which dimensions of prompt semantics predict *inter-model* reward gaps—concentrates in the leading principal components; a PCA-dimension ablation (Appendix ??) confirms the Pareto AUC surface is flat across $d \in [6, 25]$, indicating that the remaining 71% of variance encodes semantic distinctions orthogonal to routing quality. After projection, each PCA coordinate is divided by the square root of its explained variance (*whitening*), yielding approximately zero-mean, unit-variance features. This whitening step is critical for LinUCB: the isotropic prior $A_0 = \lambda I$ implicitly assumes equal feature scale across dimensions; without whitening, leading principal components dominate the UCB exploration bonus and distort arm selection. Intuitively, PCA strips away the embedding dimensions that carry noise rather than routing-relevant signal, while whitening normalises the scale of every surviving feature so that no single component dominates the bandit’s uncertainty estimates. Together, the two steps ensure that LinUCB’s exploration–exploitation balance is governed by the tunable hyperparameters ($\alpha, \gamma, n_{\text{eff}}$) rather than by artefacts of the raw feature geometry. Crucially, the PCA projection matrix is fitted *once, offline*, on $\sim 46,000$ LMSYS Arena prompts that are **strictly disjoint** from all three experimental splits (train/val/test). During all experiments the PCA transform and whitening scales are frozen and applied as inference-only projections; no component of the feature pipeline is re-fitted on the evaluation data.

5.2 Environment Construction

The $K=3$ portfolio spans three distinct cost tiers (Table 1, right). The 500 $\times$ price differential between Llama-3.1-8B and Gemini-2.5-Pro stresses the router’s ability to balance quality and cost efficiently.

We construct two distinct non-stationary test environments to evaluate the router’s robustness: 1. **Cost Drift (Price Drop)**: Simulates an API price cut. The test trajectory ($n=1,824$) is divided into three 608-prompt phases. Phase 1 uses normal pricing; Phase 2 drops Gemini-2.5-Pro’s price to \$0.0001 per request; Phase 3 restores the original pricing. Phase 3 reuses Phase 1 prompts, providing a controlled within-subject design. 2. **Catastrophic Model Failure**: Simulates a production API outage where Mistral-Large begins returning garbage responses. The test trajectory ($n=1,824$) is similarly divided into three phases of 608 prompts each. Phase 1 uses normal conditions; Phase 2 degrades Mistral-Large’s reward to 0.05 and cost to \$0 (the provider does not charge for failed requests); Phase 3 restores Mistral to normal operation (reusing Phase 1 prompts for controlled comparison). A separate reward-drift scenario (Llama–Mistral reward swap) is presented in Appendix A.

5.3 Baselines

We compare ParetoBandit against three baselines representing increasing levels of routing sophistication:

- (1) **Fixed Policy (Offline)**: Warmup priors deployed without online learning ($\gamma=1.0$, updates disabled) and a static cost penalty λ_s . The industry standard: deploy an offline-trained router with a hand-tuned cost weight.

- (2) **Naive Bandit** ($\gamma=1.0$): LinUCB with warmup priors and online learning, but infinite memory (no forgetting) and a static cost penalty. The approach taken by existing bandit-based LLM routers [22, 31, 40].
- (3) **SW-UCB** ($W=200$): Sliding-Window LinUCB [16] without warmup priors ($\alpha=0.01$). Retains only the last W observations with equal weighting. Window size $W=200$ matches ParetoBandit's asymptotic effective sample size $1/(1-\gamma)=200$ —the standard equivalence between exponential and rectangular windows—ensuring equal estimation precision while isolating the forgetting shape and initialisation as the experimental variables. Used in the appendix reward-swap experiment (Appendix A).
- (4) **ParetoBandit** ($\gamma=0.995$): Our proposed system, initialized with warmup priors, utilizing geometric forgetting, and governed by the Primal-Dual BudgetPacer targeting a specific dollar budget. All prior-initialised conditions (Fixed, Naive, ParetoBandit) share $\alpha=0.1$ and $n_{eff}=1,000$. The *prior strength* n_{eff} [18] is a robustness knob that governs how quickly online evidence can override the offline warmup priors. This tunability is essential because priors are trained on historical data that may already be partially stale by deployment time: in the LLM ecosystem, providers refresh model weights, adjust RLHF procedures, and shift pricing on timescales of weeks [7, 38], so an offline prior's quality estimates carry inherent uncertainty about their continued accuracy. n_{eff} sets the effective number of pseudo-observations the prior contributes, directly controlling how long offline estimates dominate over incoming online evidence. A high n_{eff} keeps the router close to its prior policy for many steps—stable early routing, but slow to correct if the prior is wrong; a low n_{eff} lets the router begin learning from live traffic almost immediately, acting as a safety valve against prior–reality mismatch at the cost of noisier early decisions (Appendix A.4 quantifies this interaction empirically). With $n_{eff}=1,000$ and a validation split of 1,785 prompts, the prior is influential during early deployment but transient by design: geometric forgetting ($\gamma=0.995$) gradually replaces offline estimates with online evidence over ~ 200 effective-memory steps. After sufficient online learning, the router's steady-state quality is determined by recent observations regardless of initialisation—a burn-in ablation (Appendix A.3) confirms that warmup priors alone (0% burn-in) achieve the same test regret as the full protocol (74.0 vs. 74.4, $p=0.64$), while a cold start without priors incurs +10.0% higher regret ($p < 10^{-5}$). The priors' value is thus a *deployment latency* gain: informed routing from request #1. SW-UCB uses $\alpha=0.01$ (appropriate for cold-start without priors, where no warmup information is available to stabilise early routing). Experiment 2 additionally includes a **Recalibrated Bandit** that re-tunes its static penalty at the Phase 2 boundary, serving as the strongest static-penalty baseline for cost-drift evaluation.

5.4 Metrics

We evaluate the systems using three primary metrics and four supplementary routing metrics standard in the LLM routing literature [28?]:

- (1) **Cumulative Regret**: Measured against a cost-adjusted oracle that selects the model maximizing $r_{a,t} - \lambda_s \tilde{c}_a$ at each step t (where λ_s is a shared static penalty). Lower regret indicates better routing decisions relative to the optimal choice.

Table 2: Phase 1 budget utilisation under cost drift (Experiment 2; 20 seeds). Utilisation = realised cost / target B ; values near $1.0\times$ indicate precise budget control. ParetoBandit is the only condition that reliably meets the target across all budget regimes.

Budget Target	Fixed	Naive	ParetoBandit
Tight ($\$3.0\times 10^{-4}$)	$0.51\times$	$1.52\times$	$1.00\times$
Moderate ($\$6.6\times 10^{-4}$)	$0.46\times$	$1.16\times$	$0.99\times$
Loose ($\$1.9\times 10^{-3}$)	$2.70\times$	$2.74\times$	$0.97\times$

- (2) **Budget Compliance**: The ratio of actual realized cost to the target budget B . A perfect pacer achieves a ratio $\leq 1.0\times$, maximizing quality without overspending. Ratios $> 1.0\times$ indicate budget violations.

- (3) **Pareto AUC**: The area under the quality–cost Pareto frontier, measuring the router's efficiency across all possible cost trade-offs. Higher AUC indicates better quality at lower costs.

To facilitate comparison with the LLM routing literature, we additionally report four metrics evaluated on the Pareto frontier produced by each method (Table 3):

- (1) **cost@Q% qual** ($\downarrow$): the cost on the Pareto frontier at $Q\%$ of per-prompt oracle quality (the reward obtained by always selecting the highest-quality model for each prompt). Measures how cheaply a method reaches a given quality threshold.
- (2) **qual@C% cost** ($\uparrow$): the quality achieved at $C\%$ of per-prompt oracle cost. Measures quality at a fixed cost fraction.
- (3) **Save@Q% qual** ($\uparrow$): fractional cost reduction at $Q\%$ of oracle quality relative to the oracle cost, i.e. $1 - \text{cost}@Q / \text{cost}_{\text{oracle}}$.
- (4) **AUCPC** ($\uparrow$): the normalised Area Under the Cost–Performance Curve [?], integrated over the model-cost range [cheapest model cost, most expensive model cost] and scaled to $[0, 1]$ using cheapest-model quality and oracle quality as anchors.

504.19 500.73 500.43 500.19 500.03 500.00 500.00
 90\$1.3 $\times 10^{-3}$ \$5\$4.7 $\times 10^{-3}$ 500.907250.864958.090\$4.1 $\times 10^{-4}$ 95\$4.1 $\times 10^{-3}$ 500.910250.8
 2SteadyReward0.788 2RewardGapPct8.1 2CostChangePct3.0 2SteadyRe-
 ward0.788 2RewardGapPct9.7 2CostChangePct1.5 2SteadyRe-
 ward0.801 2RewardGapPct11.2 2CostChangePct1.6 1Cost\$9.6e-
 03 2SteadyCost\$1.4e-02 2SteadyRewardShort0.914

6 Results

We evaluate ParetoBandit across four escalating experiments, moving from stationary budget pacing to complex, non-stationary environments involving cost drift and catastrophic model failure. Table 2 summarises the headline budget compliance findings from Experiment 2.

6.1 Stationary Budget Pacing

We first ask: *can the router accept a dollar budget directly and automatically achieve it, without sacrificing routing quality compared to an offline-tuned static penalty?*

We evaluate the $K=3$ portfolio under stationary conditions. Following the standard online-learning protocol, the policy continues

to adapt during the test trajectory—metrics reflect cumulative performance over the stream, not a frozen snapshot. The **Static baseline** uses a fixed cost-penalty $\lambda_s \in \{0, 0.05, \dots, 1.0\}$, producing a 7-point Pareto frontier. **ParetoBandit (BudgetPacer)** is evaluated at seven log-spaced targets, ranging from the cheapest model’s empirical mean cost (\$0.000029/req) to the most expensive (\$0.0150/req).

Pareto frontier. Both methods trace a quality–cost Pareto curve on the test set. At tight budgets (cost < \$0.001/req), the pacer achieves higher reward at lower cost: at a target of \$0.00008/req, the pacer reaches a mean reward of 0.813 ± 0.0004 . The nearest static point ($\lambda_s=1.0$) achieves only 0.804 ± 0.0003 while spending $4.4\times$ more (\$0.00037/req). The pacer dominates because its adaptive λ_t creates an implicit *spend-when-valuable, save-when-cheap* policy: when easy prompts allow cheap routing, λ_t decreases, freeing budget for harder prompts where expensive models provide greater quality uplift. The static penalty applies the same cost tax uniformly, forgoing this temporal budget allocation. In operational terms, this Pareto dominance means the pacer delivers 1.1% higher reward than the best static penalty at $4.5\times$ lower cost—a gap that widens as budgets tighten, precisely where cost control matters most.

Notably, the two frontiers differ not only in quality at shared operating points but in *coverage*: the static curve terminates at \$0.00037/req ($\lambda_s=1.0$, 93% Llama), while the pacer extends to \$0.00003/req (99.9% Llama)—a $12\times$ gap. This is structural, not a sweep artefact: for any finite λ_s , the LinUCB exploration bonus on expensive models occasionally exceeds the fixed penalty, leaking traffic to costlier arms. The adaptive λ_t faces no such ceiling—it ratchets upward until the budget constraint binds, providing continuous Pareto coverage across the full cost spectrum without offline calibration.

Figure 2 shows that ParetoBandit equipped with the BudgetPacer perfectly traces this optimal penalty Pareto frontier. At unconstrained budgets (cost > \$0.005), both methods asymptote toward a quality ceiling of ~ 0.922 —itself 95.7% of the per-prompt oracle (0.963)—and the dual variable correctly decays to zero, so the pacer gracefully becomes a no-op. The annotated oracle gaps frame the practitioner’s tradeoff concretely: a $\sim 4\times$ increase in oracle gap ($4.3\% \rightarrow 17.7\%$) buys a $300\times$ cost reduction; a more moderate 80% cost saving (\$0.0095 $\rightarrow$ \$0.0018/req) widens the gap by only 1.4 pp.

Table 3: LLM routing evaluation metrics comparing the static cost-penalty baseline and the BudgetPacer ($K=3$, 20 seeds). **cost@Q% qual:** cost on the Pareto frontier at Q% of per-prompt oracle quality (0.963). **qual@C% cost:** quality at C% of per-prompt oracle cost (\$0.00510/req). **Save@95%:** fractional cost reduction at 95% of oracle quality relative to the oracle cost. **AUCPC:** normalised area under the cost–performance curve, integrated over the model-cost range [$\$2.9 \times 10^{-5}$, $\$1.5 \times 10^{-2}$]. **Bold marks the better value in each column.** **Pareto AUC dominance:** Wilcoxon $p = 1.3e - 05$.

Method	Pareto AUC $\uparrow$	AUCPC $\uparrow$	cost@90% qual $\downarrow$	cost@95% qual $\downarrow$	qual@60% cost $\downarrow$	qual@25% cost $\downarrow$	Save@95% $\downarrow$
Static	0.9239	0.682	\$0.00132	\$0.00469	0.907	0.864	8.0%
Pacer	0.9252	0.713	\$0.000412	\$0.00409	0.916	0.891	19.7%

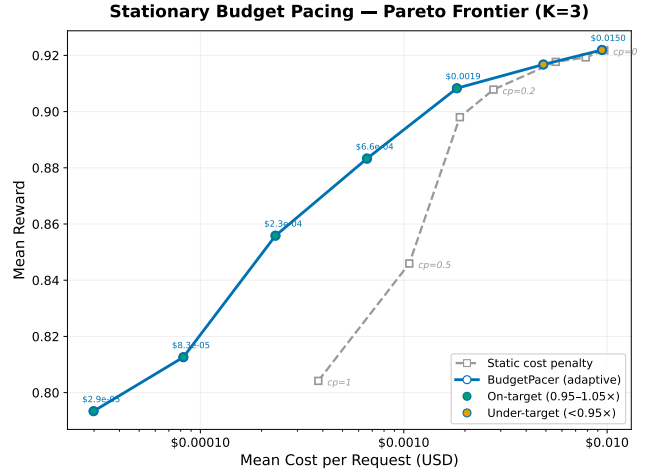


Figure 2: The Pareto frontier under stationary conditions. ParetoBandit with BudgetPacer perfectly traces the theoretical optimal curve achieved by offline λ -tuning.

Routing evaluation metrics (Table 3). All metrics use the *per-prompt oracle* (always selecting the highest-quality model for each prompt; quality 0.963, cost $\$5.1 \times 10^{-3}$ /req) as the reference. The pacer’s advantage is most visible at moderate quality thresholds: at 90% of oracle quality, the pacer reaches the frontier at 90/req versus 090– $3.2\times$ cheaper. At 95% of oracle quality (close to the unconstrained router’s own performance), the gap narrows (95 vs. 095), but the pacer still achieves 95% cost savings relative to the oracle versus 95% for Static. The normalised AUCPC confirms this advantage across the full cost range (vs. 0.682), and a Wilcoxon signed-rank test on per-seed Pareto AUC rejects the null at $p = 1.3e - 05$. Quality at a fixed oracle-cost fraction shows a sharper separation at tight budgets (qual@25% cost: 25 vs. 25) than at looser ones.

Budget compliance. The pacer’s primary advantage is its ability to hit a target. For the six tightest targets, actual spend ranges from $0.97\times$ to $1.02\times$ of the budget—effectively perfect compliance. Figure 3 illustrates the resulting steady-state model allocation under a strict budget, dynamically balancing between Llama-3.1-8B and Mistral-Large-2512 to maximize quality. The loosest target ($B=\$0.015$) shows under-utilisation ($0.64\times$) because the unconstrained router’s natural spend (\$0.0094/req) is already below this target; the pacer constrains spend *from above* but never forces the router to spend more than its quality-optimal policy requires.

6.2 Budget Pacing under Cost Drift

We now evaluate the pacer’s ability to track cost changes in real time: a three-phase cost drift while operating under strict budget constraints. The held-out test split ($n=1,824$) provides 1,216 unique prompts and a disjoint set of 608 for Phase 2 (Gemini price drop to nearly zero). Phase 3 restores the original pricing and reuses the same 608 prompts as Phase 1, making the P1 vs P3 comparison a controlled within-subject design: any difference is attributable solely to the

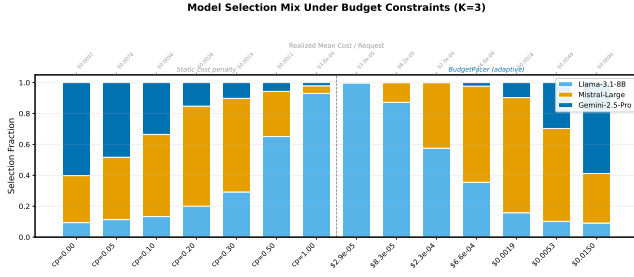


Figure 3: Model allocation mix under a strict stationary budget. The BudgetPacer stabilizes rapidly to optimal proportions.

router’s internal state (accumulated Phase 2 history), not prompt-sampling variability. With geometric forgetting $\gamma=0.995$, Phase 1 observations are down-weighted by $\gamma^{608} \approx 0.047$ by Phase 3, so the learner retains negligible memory of individual Phase 1 evaluations. The Fixed Policy (no online learning) serves as a null check—its P1 and P3 metrics are identical, confirming this control. We evaluate three budget regimes: Tight (\$0.0003/req), Moderate (\$0.00066/req), and Loose (\$0.0019/req).

Budget compliance is the key differentiator. The Fixed Policy (using an offline-tuned static penalty λ_s) is wildly inconsistent: it under-spends at tight and moderate budgets (0.51 $\times$ and 0.46 $\times$) but massively over-spends at the loose budget (2.70 $\times$). A single static penalty cannot simultaneously satisfy different budget regimes. The Naive Bandit is similarly erratic: it overshoots at tight (1.52 $\times$) and moderate (1.16 $\times$), and over-spends by 2.74 $\times$ at the loose budget because the static penalty cannot adjust to the actual cost landscape. To isolate the value of *continuous* online tracking from merely reacting to a price change, we add a **Recalibrated Bandit**: identical to the Naive Bandit in Phase 1, but at each phase boundary its static penalty is re-tuned offline via a grid search on the validation split with the new phase’s pricing. This is the strongest static-penalty baseline, granting oracle knowledge of both change points and two offline calibration passes. Even so, the Recalibrated Bandit inherits Phase 1 non-compliance and cannot track intra-phase spending dynamics (Table 4).

ParetoBandit is the only system that reliably hits the budget in Phase 1 (0.97 $\times$ to 1.00 $\times$ across all three targets; Table 2b). The BudgetPacer’s primal-dual mechanism adjusts λ_t continuously based on observed costs, providing closed-loop control that no static penalty—even with periodic recalibration—can replicate. From an SLA perspective, these deviations translate directly to financial risk: the Fixed Policy’s 2.70 $\times$ overshoot at the loose budget means spending more than 160% above the agreed-upon target—an overrun that grows linearly with traffic volume—while its 0.51 $\times$ utilisation at the tight budget means leaving roughly half the quality headroom on the table.

Exploiting the price drop (Phase 2). When Gemini becomes nearly free in Phase 2, the BudgetPacer automatically detects the cost reduction via its EMA signal. The three budget regimes produce qualitatively distinct—but equally principled—dual-variable responses (Figure 4). At all three targets, post-drop costs fall below the budget,

so λ_t decays and the constraint becomes slack. The decay speed depends on the margin: at moderate and loose targets (0.53 $\times$ and 0.19 $\times$ respectively), λ_t drops to near-zero and the router reverts to quality-only optimization, increasing Gemini selection to ~46–51%. At tight target, the margin is narrower (0.95 $\times$), so λ_t decays from 0.41 to ~0.07—slower, but in the same direction. The tight condition still achieves the highest Gemini adoption (72%) of any constrained condition, because the residual λ_t provides mild cost pressure favoring the now-cheapest arm (Gemini, $\tilde{c}\approx 0$) over Mistral ($\tilde{c}\approx 0.49$). All three regimes show highly significant reward lifts ($\Delta = +0.024$ to $+0.071$; $p < 10^{-3}$), with the tight budget exhibiting the largest gain because its Phase 1 constraint was most binding, creating the most room for improvement when costs collapse.

Recovering after price restoration (Phase 3). When Gemini’s original pricing is restored at step 1217, the BudgetPacer reverses course: λ_t rises back toward Phase 1 levels as the cost EMA exceeds the budget target, and Gemini adoption drops to near-zero. At tight budget, λ_t rises from 0.07 to ~0.51; at moderate from 0.01 to ~0.40; at loose from near-zero to ~0.10 (Figure 4b). Phase 3 λ_t does not exactly match Phase 1 values (e.g., tight: 0.51 vs. 0.41)—this asymmetry reflects the ramp-up transient from near-zero through the slack region and is consistent with the finite 608-step phase length. Budget compliance recovers to 1.01 $\times$, 1.02 $\times$, and 1.00 $\times$ across the three targets—within 2% of the budget at all levels. No other condition achieves this round-trip compliance: the Recalibrated Bandit obtains a second oracle recalibration at the Phase 3 boundary yet still under-spends (0.66 $\times$ –1.02 $\times$), while the Naive Bandit cannot recover at moderate and loose budgets (0.89 $\times$ and 2.98 $\times$). This Phase 3 recovery is the strongest evidence that the primal-dual mechanism provides genuine closed-loop budget control rather than merely reacting to favorable cost changes.

Quality cost of the pacer. An unconstrained ParetoBandit baseline (same $\gamma=0.995$, no pacer) isolates the pacer’s quality impact. In Phase 2, the unconstrained baseline reaches reward 0.928; ParetoBandit at moderate/loose budgets reaches 0.927—a gap of $\leq 0.1\%$. The pacer itself is nearly free in quality terms. This quality difference is the cost of non-stationarity robustness—validated here by the reward lifts that the forgetting factor enables.

The cost–quality tradeoff under normal operation. Phase 1 (before any cost perturbation) reveals the practical value of the dual-variable mechanism under stable conditions. In this phase, the unconstrained baseline spends \$9.5e-03/req and achieves reward 0.9158. The loose budget constraint (\$1.9 $\times 10^{-3}$ /req target) achieves reward 0.9026—only 1.4% below unconstrained—while spending 5.3 $\times$ less, a 81% cost reduction. The pacer accomplishes this by routing the majority of traffic through Mistral-Large (76%) and Llama-8B (15%), reserving Gemini-Pro (9%) for prompts where its quality advantage justifies the cost. Tighter budgets push the trade-off further: at moderate, cost drops 93% for a 4.2% quality reduction; at tight, 97% savings for a 6.6% quality gap. This gradient is the pacer operating as designed: even a generous constraint avoids the unconstrained router’s tendency to over-route to expensive models when cheaper alternatives are nearly as good for the given context, while tighter constraints progressively restrict access to high-cost arms. In production settings, these savings compound

Table 4: Budget compliance under cost drift (Experiment 2, 20 seeds, three phases). Each cell shows realised average cost as a multiple of the budget target ($1.00\times$ = perfect). Bold marks values within 5% of $1.00\times$. † Phase 2 constraint non-binding: the price drop reduces all methods’ costs below target, regardless of algorithm. ParetoBandit is the only condition that reliably meets the target in Phase 1 and recovers compliance in Phase 3 after the price is restored.

Budget	Condition	Phase 1	Phase 2	Phase 3
Tight	Fixed Policy	$0.51\times$	$0.89\times^\dagger$	$0.51\times$
	Naive Bandit	$1.52\times$	$0.89\times^\dagger$	$1.11\times$
	Recalibrated	$1.52\times$	$0.99\times$	$0.66\times$
	ParetoBandit	$1.00\times$	$0.95\times$	$1.01\times$
Moderate	Fixed Policy	$0.46\times$	$0.40\times^\dagger$	$0.46\times$
	Naive Bandit	$1.16\times$	$0.40\times^\dagger$	$0.89\times$
	Recalibrated	$1.16\times$	$0.45\times^\dagger$	$0.99\times$
	ParetoBandit	$0.99\times$	$0.53\times^\dagger$	$1.02\times$
Loose	Fixed Policy	$2.70\times$	$0.14\times^\dagger$	$2.70\times$
	Naive Bandit	$2.74\times$	$0.14\times^\dagger$	$2.98\times$
	Recalibrated	$2.74\times$	$0.19\times^\dagger$	$1.02\times$
	ParetoBandit	$0.97\times$	$0.19\times^\dagger$	$1.00\times$

with traffic volume—a deployment routing one million requests per month would reduce costs from $\sim\$9,400$ (unconstrained) to $\sim\$1,800$ (loose) with negligible user-perceptible quality difference.

Tight budgets: full round-trip adaptation. The tight target ($B=\$0.0003/\text{req}$) is the most instructive case because it transitions from *binding* to *slack* and back to *binding* across the three phases. In Phase 1, λ_t averages 0.41, enforcing a Llama/Mistral mix; after the price drop, λ_t decays to ~ 0.07 as costs fall below target. The tight condition achieves the highest Gemini adoption (72%) of any constrained condition. In Phase 3, when original pricing is restored, λ_t rises to ~ 0.51 and Gemini selection returns to near-zero, recovering budget compliance at $1.01\times$. This full round-trip—binding $\rightarrow$ slack $\rightarrow$ binding—demonstrates that the primal-dual mechanism adapts in both directions without operator intervention. The tight budget also exhibits the largest reward lift ($\Delta = +0.071$) because its Phase 1 constraint was most binding, creating the most room for improvement when costs collapse.

Ecological validity of the cost perturbation. The near-total price collapse used in Phase 2 is deliberately extreme: it stress-tests the pacer at the boundary of its operating range, where the dual variable must traverse from binding to fully slack and back. Under milder cost perturbations, the same EMA-driven dual update produces proportionally smaller λ_t adjustments with faster convergence, since the required policy shift is smaller—the mechanism’s response is continuous and monotonic in the magnitude of the cost signal. Importantly, discrete step-change pricing is the norm in production LLM deployments, not the exception: OpenAI reduced GPT-4 Turbo input prices by 67% in a single announcement [29], and Google cut Gemini 1.5 Flash input pricing by 78% overnight [17]. The experimental design therefore mirrors the structure of real-world cost shocks—abrupt, provider-initiated, and often large—while testing at an extreme that bounds the pacer’s worst-case adaptation latency.

6.3 Resilience to Catastrophic Model Failure

Having established the pacer’s precision under cost drift, we now stress-test the system against the most severe non-stationarity a production router can face: a *catastrophic model failure* where a previously top-performing model begins returning garbage responses. Cloud API outages, 5xx errors, timeout cascades, and silent quality degradation after provider-side model updates are routine in production multi-LLM deployments. When such a failure occurs, a static router continues sending traffic to the dead model—degrading user experience and wasting latency—until a human detects the problem, re-evaluates alternatives, and redeploys. We ask: *can ParetoBandit detect a catastrophic failure through its reward signal alone, redistribute traffic across the remaining $K>2$ models, and maintain budget compliance throughout?*

Setup. We construct a three-phase scenario on the $K=3$ portfolio using the held-out test split ($n=608$ prompts per phase):

- **Phase 1** (steps 1–608): Normal operation. All models are healthy; the router has converged via online training on the validation split.
- **Phase 2** (steps 608– 2×608): **Catastrophic failure**—Mistral-Large’s reward drops to 0.05 (the API returns errors or garbage) and per-request cost drops to $\$0$ (the provider does not charge for failed requests). The router must detect the quality collapse through its reward signal and redistribute traffic to Llama-8B and Gemini-Pro.
- **Phase 3** (steps 2×608 – 3×608): **Recovery**—Mistral-Large is restored to normal quality and pricing. Phase 3 reuses Phase 1 prompts so that any P1-vs-P3 difference is attributable solely to the router’s internal state, not prompt-sampling variability.

The model registry is *not* mutated at phase boundaries: the router’s internal cost penalty continues to reflect Mistral’s normal pricing. Adaptation is driven purely by the observed reward signal, as it would be in production.

Why these baselines. Three algorithm variants are compared at each of three budget targets (Tight, Moderate, Loose), plus an unconstrained baseline (20 seeds): (1) **Fixed Policy**: frozen warmup priors, static cost penalty, no online learning—the counterfactual that keeps routing to the dead model; (2) **Naive Bandit** ($\gamma=1.0$): online learning with infinite memory and a static cost penalty—the approach taken by existing bandit-based LLM routers [31, 40]; (3) **ParetoBandit** ($\gamma=0.995$): geometric forgetting with an active BudgetPacer—the proposed method.

Results (Figure 5; Table 5). The three conditions produce starkly different failure responses:

- **Fixed Policy** cannot learn; it continues routing 0.3752-quality traffic through Mistral at 61% allocation (moderate budget), suffering the most severe reward collapse. At the loose budget, where the static penalty favours Mistral most heavily, the Fixed Policy routes 70% of traffic to the dead model, driving Phase 2 reward down to 0.3157.
- **Naive Bandit** ($\gamma=1.0$) detects the failure and reduces Mistral traffic, but without a BudgetPacer its budget explodes: the static cost penalty that was calibrated for Phase 1 pricing cannot adjust to the changed cost landscape. At the moderate budget, the Naive

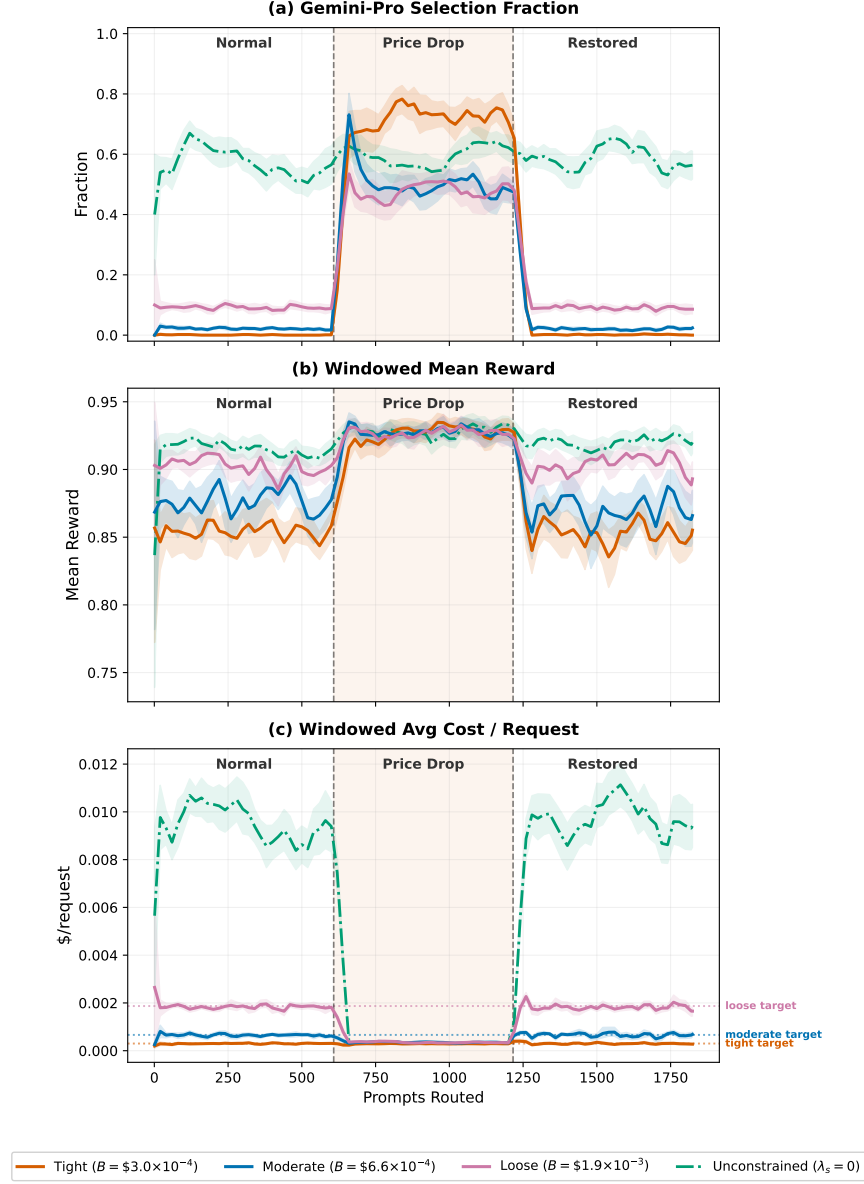


Figure 4: ParetoBandit adaptation dynamics across a three-phase cost scenario: normal pricing → Gemini price drop (step 609) → price restored (step 1217). $K=3$, 20 seeds, 95% bootstrap CI. (a) Gemini-Pro selection fraction: when Gemini becomes nearly free, the tight budget surges to ~70% Gemini adoption while the unconstrained baseline reaches ~60%—the pacer exploits the price drop more aggressively at tight budgets because the residual λ_t favours the now-cheapest arm. (b) Windowed mean reward (trailing 50 steps): quality lifts during the price drop track the Gemini adoption in (a), then return to Phase 1 levels after price restoration. (c) Windowed average cost per request; dotted lines mark budget targets. All three regimes recover budget compliance in Phase 3. See Table 4 for quantitative comparisons.

Bandit overshoots by $7.46\times$ in Phase 2 and $9.63\times$ in Phase 3—a persistent budget violation that worsens over time as online learning shifts traffic toward expensive arms without cost feedback.

- **ParetoBandit** ($\gamma=0.995$) detects the failure, redistributes traffic (Mistral drops to 8% at moderate budget), and maintains budget

compliance: $1.00\times$, $1.00\times$, and $0.97\times$ across the three phases. The BudgetPacer’s dual variable λ_t adjusts automatically as the cost landscape changes, providing closed-loop cost control that the Naive Bandit’s static penalty structurally cannot achieve.

Budget compliance is the key differentiator. The most striking result is not the reward comparison but the budget compliance

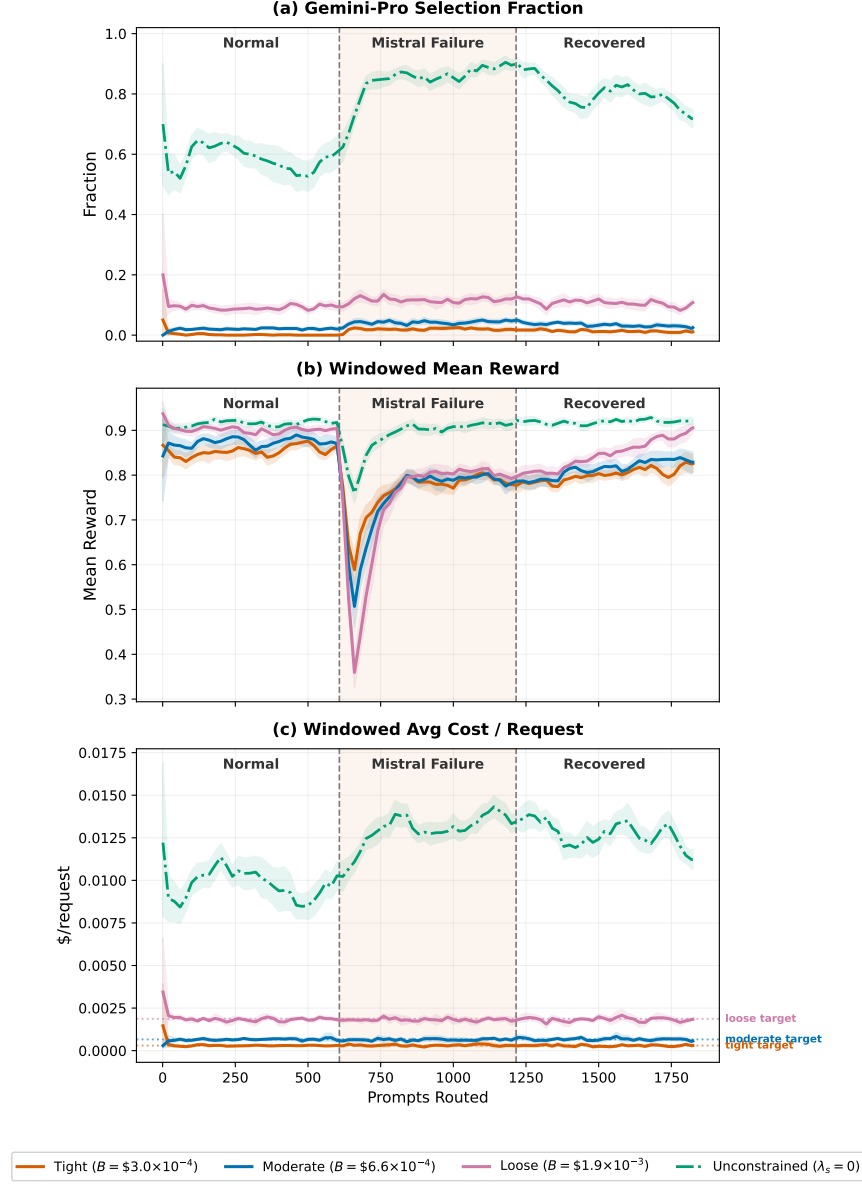


Figure 5: ParetoBandit adaptation dynamics under catastrophic model failure: normal $\rightarrow$ Mistral failure (reward $\rightarrow$ 0.05, cost $\rightarrow$ \$0, step 608) $\rightarrow$ recovered (step 2×608). $K=3$, 20 seeds, 95% bootstrap CI. (a) Gemini-Pro selection fraction: the unconstrained baseline freely shifts $\sim 80\%$ of traffic to expensive Gemini-Pro when Mistral fails, while budget-constrained conditions remain near zero—forced to absorb the failure via cheaper Llama-8B. (b) Windowed mean reward (trailing 50 steps): the Gemini gap in (a) explains the quality gap—unconstrained recovers to ~ 0.9 during failure while constrained conditions dip more because Llama is lower quality. (c) Windowed average cost per request; dotted lines mark budget targets. All three budget regimes maintain compliance ($0.97\text{--}1.03\times$) throughout the failure, while unconstrained cost spikes. See Table 5 for quantitative comparisons across algorithms.

column in Table 5. ParetoBandit maintains budget ratios within $0.97\text{--}1.03\times$ across *all* nine phase-budget combinations. The Naive Bandit, despite detecting the failure and recovering reward, violates the budget by up to $7.58\times$ (tight, Phase 3)—a persistent overrun that grows linearly with traffic volume. This demonstrates that online

learning alone is insufficient for budget-constrained non-stationary routing: the dual-variable mechanism is essential for maintaining cost invariance through disruptions.

Adaptation dynamics and the cost-quality tradeoff. Figure 5 reveals the mechanism behind budget compliance. When Mistral fails,

Table 5: Failure response across algorithm variants (Experiment 3, 20 seeds). Phase 1: normal operation; Phase 2: Mistral-Large catastrophic failure (reward $\rightarrow$ 0.05, cost $\rightarrow$ \$0); Phase 3: model restored. Bold marks budget compliance within 5% of target. ParetoBandit detects the failure, redistributes traffic, and maintains budget compliance; Fixed Policy keeps routing to the dead model. Phase 2 budget ratio may fall below 1.00 $\times$ because the failed model’s cost drops to \$0.

Budget	Algorithm	Mean Reward			P2 Mistral-Large
		P1	P2	P3	
3*Tight (3.0×10^{-4})	Fixed Policy	0.831	0.590	0.831	31%
	Naive Bandit	0.863	0.688	0.809	19%
	ParetoBandit	0.854	0.760	0.802	6%
3*Moderate (6.6×10^{-4})	Fixed Policy	0.872	0.375	0.872	61%
	Naive Bandit	0.890	0.677	0.861	25%
	ParetoBandit	0.874	0.750	0.816	8%
3*Loose (1.9×10^{-3})	Fixed Policy	0.928	0.316	0.928	70%
	Naive Bandit	0.920	0.816	0.916	13%
	ParetoBandit	0.901	0.736	0.849	11%

reward collapses within ~ 52 steps as the 50-step trailing window fills with near-zero rewards. At moderate budget, the trough reaches 0.507 (a 41.9% drop), with deeper troughs at looser budgets (0.359, -60.2% for loose) because these conditions route more traffic to Mistral pre-failure. ParetoBandit detects the collapse and adapts: reward stabilises within ~ 172 steps (moderate) to ~ 192 steps (loose), reaching Phase 2 steady-state levels of $2\text{SteadyReward} - 2\text{SteadyReward}$ across budget regimes.

The unconstrained baseline reveals the cost of this stability. Without a BudgetPacer, the unconstrained router freely shifts $\sim 80\%$ of traffic to Gemini-Pro—the remaining high-quality model—and recovers reward to $2\text{SteadyRewardShort}$ during the failure (Figure 5a). However, this comes at a +45.4% cost spike ($1\text{Cost}/\text{req} \rightarrow 2\text{SteadyCost}/\text{req}$; Figure 5c). Budget-constrained conditions cannot follow this strategy: their BudgetPacer penalises expensive Gemini-Pro traffic, forcing adaptation through cheaper Llama-8B and yielding a $2\text{RewardGapPct} - 2\text{RewardGapPct}\%$ quality gap relative to Phase 1. The tradeoff is principled: cost compliance costs $\sim 10\%$ quality during a catastrophic failure, while the unconstrained alternative maintains quality at the price of a 45% cost overrun—a budget violation that in production would breach SLAs and accumulate linearly with traffic volume.

Recovery dynamics. When Mistral is restored in Phase 3, the geometric forgetting factor ($\gamma=0.995$) allows ParetoBandit to re-discover the recovered model via UCB exploration as the Phase 2 failure memory is discounted. Phase 3 reward approaches but does not fully match Phase 1 levels within the 608-step phase length—consistent with the ~ 200 -step effective memory window requiring time to accumulate sufficient positive evidence for Mistral. The BudgetPacer tracks this transition smoothly, with λ_t adjusting to maintain compliance as the traffic mix evolves.

6.4 Hyperparameter Selection via Epsilon-Constraint

Selecting hyperparameters ($\alpha, n_{\text{eff}}, \gamma$) for a non-stationary router poses a fundamental tension: standard cross-validation maximises stationary quality and selects $\gamma=1.0$ (no forgetting) for both variants, yet Section 6.3 showed that infinite memory fails to adapt under distribution shift. The core idea of our resolution is simple: first, identify all hyperparameter settings that route well under normal (stationary) conditions; then, among those near-optimal settings, pick the one that adapts fastest when the environment changes—**Budget Ratio ($\times$ target)** which would select settings that catastrophically fail under drift. We formalise this with the **epsilon-constraint method**, a principled bi-objective selection procedure. Two objectives. Each configuration in the grid ($6 \times 6 \times 4$ values of $\alpha, n_{\text{eff}}, \gamma$) is scored on two metrics using the validation split: (1) **Budget-paced Pareto AUC** (primary): the area under the quality–cost Pareto frontier when sweeping budget targets with the BudgetPacer active. Higher is better. (2) **Phase-2 cumulative regret** (secondary): a two-phase simulation where, at the midpoint, two arms swap their reward *and* cost distributions. Only post-swap regret is recorded, averaged over all $\binom{K}{2}$ swap pairs and 10 seeds. Lower is better.

Selection rule. Among all configurations whose budget-paced AUC is within $\epsilon=0.25\%$ of the best, we select the one with the lowest Phase-2 regret. This gives budget-pacing explicit priority—no configuration is chosen that materially degrades stationary routing—while using adaptation speed as the tiebreaker within the near-optimal band. The tolerance is calibrated to the observed AUC variation: the entire 144-configuration ParetoBandit grid spans only 0.39% in budget-paced AUC, so $\epsilon=0.25\%$ covers roughly two-thirds of that range—tight enough to exclude materially worse configurations (32% of the grid), yet permissive enough to admit a diverse set of (α, γ) combinations for the secondary objective to discriminate.

Result. Figure 6 visualises the tradeoff. Configurations with $\gamma=1.0$ (blue) achieve the highest AUC but the worst post-swap regret: they cannot shed stale evidence. Lower γ values shift points downward (better adaptation) at marginal AUC cost. The ϵ -feasible band retains 98 of 144 ParetoBandit configurations (68%), excluding primarily those with aggressive forgetting or very high exploration that trade too much stationary quality. Within this feasible set, Phase-2 regret breaks ties. The method selects $\gamma=0.995$ (star) with $\alpha=0.10$, achieving Phase-2 regret of 48.4 while sacrificing only 0.24% of stationary AUC ($0.931 \rightarrow 0.929$). The AUC-only winner ($\gamma=1.0$, red X) selects $\alpha=0.01$ with minimal exploration—exactly the configuration that fails under drift. This ~ 200 -step effective memory window is long enough for stable exploitation, short enough to track realistic cost and quality shifts.

On the held-out test split the selected configurations achieve Pareto AUC of 0.9248 (ParetoBandit) and 0.9248 (Tabula Rasa) versus 0.9255 for the fixed-model envelope—deltas of -0.07% and -0.07% , both within one standard deviation of seed variance (± 0.0011 and ± 0.0011). The val-selected hyperparameters thus transfer to test without statistically significant degradation, consistent with

the ϵ -constraint design which permits up to 0.25% AUC sacrifice in exchange for adaptation capability.

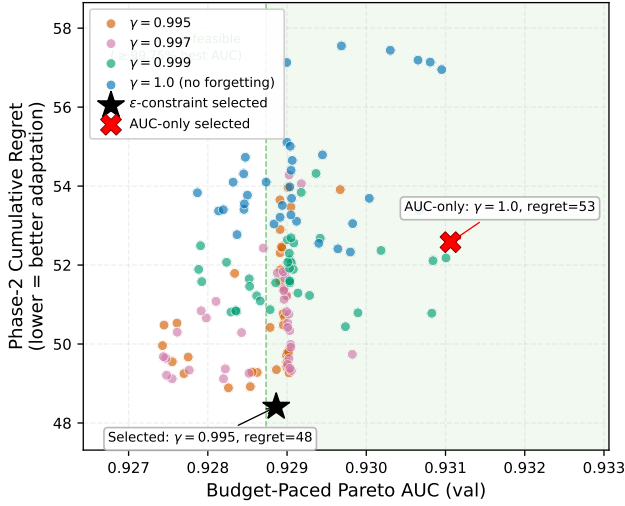


Figure 6: Efficiency-adaptability tradeoff for ParetoBandit ($K=3$, PCA-25). Each point is one $(\alpha, n_{\text{eff}}, \gamma)$ configuration; colour encodes γ . The green band marks the ϵ -feasible region ($\geq 99.75\%$ of best AUC); 98 of 144 configurations pass. The black star is the ϵ -constraint winner; the red X is the AUC-only winner ($\gamma=1.0$), which suffers the highest Phase-2 regret.

6.5 Cold-Start Model Onboarding

The experiments above assume a fixed $K=3$ portfolio. In practice, providers release new models frequently, and platform teams need to integrate them without offline evaluation or policy retraining. In Appendix A.5 we evaluate this scenario: after Phase 1 learning on the $K=3$ portfolio, Gemini-2.5-Flash is added via a single `register_model()` call with *no warmup priors*—it receives only a neutral T-shirt prior ($A=\lambda I$, $b=0$), the coldest possible LinUCB initialisation.

Despite this cold start, ParetoBandit discovers and adopts Flash in *every* trial (80/80 across 20 seeds $\times$ 4 budget tiers), reaching sustained adoption within ~ 233 post-onboarding steps. LinUCB’s exploration bonus—maximal for the uninformed arm—drives initial exploration, and the online learner quickly calibrates Flash’s quality-cost profile from live traffic. Crucially, the BudgetPacer maintains compliance throughout the portfolio expansion without any reconfiguration. The displacement analysis reveals economically rational behaviour: under budget pressure, Flash takes share from Mistral-Large (similar quality, lower cost); unconstrained, it displaces Gemini-Pro (near-Pro quality at $10\times$ lower cost).

No historical labeled data required. A supervised router requires a *pre-collected evaluation dataset* for every model in the portfolio: responses must be gathered on a representative prompt set, labeled by human annotators or an LLM judge, and the routing policy retrained on the expanded dataset—before the new model can receive any live traffic. The bandit eliminates this offline data-collection

campaign entirely. It does require a reward signal, but that signal comes from the *same* evaluation mechanism (judge, user ratings, or implicit behavioural composites; see Section 7) already serving the existing portfolio. When exploration routes a prompt to the new model, the reward pipeline evaluates its response at zero marginal data-collection cost. This distinction also avoids two risks inherent to offline evaluation: *distribution mismatch*, where a curated eval set may not reflect the live prompt distribution, and *dataset staleness*, where the eval ages as deployment patterns drift. The bandit’s “training data” is always the current production distribution, ensuring that the quality-cost profile it learns for the new model is representative from the start.

Full results and figures are in Appendix A.5.

7 Discussion and Production Implications

ParetoBandit bridges the gap between theoretical contextual bandit algorithms and the practical realities of deploying multi-LLM routers in production environments. We highlight several key implications for engineering teams and address current limitations.

7.1 Practical Impact

Interpretable Interface and SLA Compliance. The standard approach to cost-aware routing uses a static penalty λ , which requires translating a dimensionless weight into a dollar outcome via offline simulation. This mapping breaks whenever the portfolio changes. ParetoBandit’s Primal-Dual BudgetPacer accepts a dollar-denominated target (e.g., $B = \$0.002/\text{request}$) directly. This target is portable across portfolios because the normalized dual update ($\tilde{c}_t/B$) adapts the penalty λ_t to the realized cost scale automatically.

Furthermore, the pacer’s precise budget utilization ($0.97\times$ to $1.02\times$) allows it to underpin strict Service Level Agreements (SLAs) for cost (e.g., “average spend will not exceed $1.10\times B$ over any rolling window”). Static penalties offer no such guarantee, as realized costs fluctuate arbitrarily if prompt distributions drift.

Eliminating Offline Recalibration. In a deployment where model pricing changes frequently (as is common with frontier APIs), a static penalty becomes stale, requiring costly offline retraining to discover the new optimal λ . The BudgetPacer automatically detects price drops via its online cost EMA and dynamically re-allocates freed budget to higher-quality models. This autonomous optimization eliminates the need for manual intervention, maximizing application quality while strictly obeying financial constraints.

Operational Value beyond Static Penalties. Any cost-aware router reduces spend by shifting traffic to cheaper models; the relevant question is what ParetoBandit delivers *beyond* a well-tuned static penalty. Three differentiators emerge from the experiments. *Budget precision.* Across nine budget-phase conditions (Table 4), static baselines deviate from target by $0.51\times$ – $2.74\times$; ParetoBandit holds within $0.97\times$ – $1.01\times$. At a deployment routing 100,000 requests/day, the Fixed Policy’s $2.70\times$ overshoot at the loose budget ($B=\$0.0019/\text{req}$) translates to over \$300/day in SLA-violating overruns; its $0.51\times$ under-spend at the tight budget wastes quality headroom that could have improved user experience. *Autonomous adaptation.* When Gemini-2.5-Pro’s price dropped mid-cycle, ParetoBandit detected the shift and re-routed 72% of traffic to the newly cost-effective

frontier model within hundreds of requests, delivering a +0.071 quality lift at the tight budget—an improvement that static systems can replicate only through an offline recalibration cycle. *Pareto efficiency at tight budgets.* At a budget target of \$0.00008/req, ParetoBandit achieves reward 0.813 ± 0.0004 while the nearest static point ($\lambda_s=1.0$) spends $4.5\times$ more for only 0.804 ± 0.0003 —the adaptive λ_t creates a *spend-when-valuable, save-when-cheap* policy that a fixed penalty structurally cannot implement. Aggregated across the full Pareto frontier (Table 3), the pacer achieves 95% cost savings at 95% of oracle quality versus 95% for the static baseline, with AUCPC of versus 0.682.

Portfolio and Encoder Portability. The experiments in this paper evaluate a fixed configuration (MiniLM-L6 encoder, $K=3$ portfolio, shipped warmup priors), but production teams operate diverse model portfolios that evolve over time—models are added when new APIs launch, removed when deprecated, and swapped when pricing changes. A router that is hard-wired to a fixed encoder or model set forces a full retrain whenever the portfolio changes, negating the benefit of online adaptation.

ParetoBandit decouples each component of the pipeline so that every part can be replaced independently:

- **Encoder:** Teams with domain-specific embeddings (e.g., code, biomedical, multilingual) can supply any Callable[[str], np.ndarray] as a custom encoder, or pass pre-computed vectors directly, without depending on sentence-transformers.
- **PCA and warmup priors:** A two-function calibration API (`train_pca`, `generate_warmup_priors`) produces compatible artifacts for any encoder–portfolio combination from a labelled reward dataset. This lets a team onboard a new portfolio in minutes rather than days—encode representative prompts, collect reward labels, and generate priors—without modifying the routing algorithm.
- **Model portfolio:** Models can be added or removed at runtime via `register_model()` and `remove_model()` without restarting the service. New arms start from the isotropic prior ($A=\lambda I$, $b=0$) and begin learning immediately; removed arms are garbage-collected from the bandit state. Appendix A.5 validates this capability empirically: Gemini-2.5-Flash is onboarded mid-trajectory with zero warmup priors, yet LinUCB’s exploration bonus discovers and sustains adoption within ~ 233 steps across all 80 trials, while the BudgetPacer maintains cost compliance throughout the portfolio expansion.

This portability matters because the most common deployment failure mode for adaptive routers is not algorithmic—it is operational friction: the effort required to adapt the system when the model landscape changes. By making every artifact regenerable and every component swappable, ParetoBandit ensures that the routing algorithm remains the stable core while the surrounding model ecosystem evolves freely.

7.2 Limitations and Deployment Considerations

Warmup Priors: Transient Benefit, Not Persistent Advantage. The burn-in ablation (Appendix A.3) reveals that warmup priors are a *deployment latency optimisation*, not a persistent quality advantage. With geometric forgetting ($\gamma=0.995$), the prior’s contribution decays to $< 1\%$ within $\sim 1,000$ steps; after sufficient online learning, the router converges to the same steady-state regardless of initialisation

(74.0 vs. 74.4 test regret, $p=0.64$). The priors’ measurable value is concentrated in the cold-start transient: without priors, a Tabula Rasa start incurs +10.0% higher test regret ($p < 10^{-5}$), driven almost entirely by the first ~ 200 steps where the UCB exploration bonus dominates uninformed routing. This means the case for warmup priors scales with deployment sensitivity to early-stage regret—applications serving thousands of requests in the first minutes of deployment benefit substantially; systems that can tolerate a learning phase may forgo priors entirely.

Under budget constraints, the BudgetPacer partially compensates for the absence of priors by constraining the cost of uninformed exploration. The ablation decouples pacer calibration from reward-model burn-in (all conditions start with an identically pre-calibrated λ_t ; see Appendix A.3), and after this decoupling the tight-budget cold-start penalty is +6.7% ($p=0.002$), significant even after Bonferroni correction ($p_{\text{adj}}=0.006$)—compared with +10.0% in the unconstrained case. At moderate budgets, the gap shrinks to +4.1% ($p=0.017$), which does not survive Bonferroni correction ($p_{\text{adj}}=0.051$): the budget constraint narrows the feasible arm set, reducing—but not eliminating—the damage of uninformed exploration. Cost compliance is consistent across all conditions (97.6%–100.0% of target), confirming that the BudgetPacer maintains closed-loop cost control regardless of initialisation. In operational terms, teams deploying under budget constraints can rely on the pacer itself as a partial cold-start safety net; warmup priors remain valuable primarily for unconstrained or latency-sensitive deployments.

Appendix A.4 shows that the dependence on priors is safe when priors are directionally correct (even domain-mismatched priors trained on a single benchmark reduce regret), but systematically inverted priors cause harm that scales with n_{eff} —up to +13% regret at $n_{\text{eff}}=1,000$ ($p < 10^{-6}$). A conservative n_{eff} (10–100) limits worst-case exposure; future work could additionally investigate targeted exploration strategies (e.g., small ϵ -greedy budget reserves) that periodically evaluate under-explored arms to maintain calibrated quality estimates independent of the prior.

Static Unit-Cost Normalization. The normalized cost $\tilde{c}_a$ in Equation 3 and the hard ceiling filter both use a static blended rate $(\text{input_cost} + \text{output_cost})/2$ rather than weighting by the actual input/output token counts of each request. For workloads with highly asymmetric token ratios (e.g., retrieval-augmented generation with $\gg 10\times$ more input than output tokens), a token-weighted cost would give a more precise per-request estimate. In practice, this does not affect our experimental conclusions for three reasons: (i) the Pareto frontiers are evaluated using *exact per-request costs* from the offline dataset, not the router’s blended estimate; (ii) the log-scale normalization preserves the cost *ranking* of models across any reasonable token ratio—empirical validation on 1,785 prompts confirms the heuristic ordering matches true per-request cost ordering on 99.9% of prompts for $K=3$ (Appendix A.6, Figure 16); and (iii) the bandit’s learned quality model is independent of the cost penalty— $\tilde{c}_a$ biases arm selection but does not distort the reward posterior. A production deployment serving heterogeneous workloads (e.g., both summarization and RAG) could benefit from passing the expected token ratio at routing time; the open-source release already accepts `input_tokens/output_tokens` parameters and uses them for post-hoc cost logging.

Rate-Constraint Tradeoffs and Budget Sizing. Because the budget constraint is a rate target enforced via an EMA-smoothed dual variable rather than a cumulative knapsack, the system reacts to sustained cost changes with a lag determined by the EMA half-life (~ 14 observations at $\alpha_{\text{ema}}=0.05$) and the dual step size ($\eta=0.05$). A sudden, sustained shift toward expensive-model usage—e.g., a burst of hard prompts that all require frontier calls—can temporarily push realized average spend above the target B until λ_t corrects. In practice, the ADAPTIVE mode’s hard ceiling acts as a circuit breaker, bounding worst-case per-request cost even before λ_t adjusts.

This transient overshoot should be weighed against the failure mode of the alternative. A finite-horizon knapsack (e.g., PILOT’s approach [31]) provides a hard cumulative guarantee: total spend never exceeds B_{total} . However, if request volume exceeds the anticipated horizon T , the knapsack exhausts its budget early, forcing the system to either refuse service or fall back to the cheapest model for the entire remaining tail of the horizon—degrading quality for potentially thousands of requests at precisely the moment demand is highest. The rate constraint has no such cliff: it absorbs volume variability gracefully, self-correcting within a bounded number of requests regardless of total traffic volume.

Eliminating the Rolling-Horizon Dependency. Production budget-pacing systems in adjacent domains—most notably digital advertising—have long recognized the horizon limitation of cumulative knapsacks and address it through *receding horizon control* (RHC), also known as model predictive control: at each update interval, the system re-forecasts remaining traffic volume, recomputes the budget allocation over the shrinking horizon, and adjusts bid multipliers accordingly [9]. While effective in high-volume advertising markets with well-characterized traffic patterns, RHC couples budget compliance to a separate traffic-forecasting pipeline whose prediction errors propagate directly into budget violations—overestimating remaining volume causes the very premature budget exhaustion the knapsack was designed to prevent, while underestimating it leaves quality-improving frontier calls on the table. An LLM routing deployment adopting a knapsack formulation would inherit this dependency, requiring a traffic-forecast model that must itself be maintained, retrained on evolving request patterns, and monitored for drift—doubling the operational surface area of the system.

ParetoBandit’s rate-constraint formulation eliminates this dependency entirely. Because the dual update targets the running average cost rather than a depleting budget, the system requires no estimate of remaining traffic: it self-regulates at any volume, whether the next hour brings 100 requests or 100,000. This makes the router fully self-contained—a single stateless update rule ($\lambda_{t+1} \leftarrow [\lambda_t + \eta(\bar{c}_t/B - 1)]^+$) replaces the forecast–replan–execute loop that RHC demands—reducing deployment from a coupled pair of ML systems (traffic forecaster + budget controller) to a single adaptive controller with no external dependencies. For engineering teams, this distinction is operationally significant: the rate constraint eliminates an entire failure mode (forecast-induced budget misallocation) and removes a maintenance burden (the forecasting pipeline) that is orthogonal to the routing problem itself.

We recommend setting the budget target B at a per-request rate where each routed response is individually profitable—i.e., the business value derived from each response exceeds B . Under this

operating regime, an unexpected surge in traffic is not a risk to be mitigated but a *welcome* event: every additional request generates net-positive revenue at the same per-request cost, and the dual variable maintains budget compliance automatically without any reconfiguration. Transient overshoot during the spike remains within the margin of per-request profitability—the operator may temporarily spend slightly above B per request, but every request is still net-positive, and λ_t corrects within tens of observations. By contrast, a knapsack under the same spike exhausts its cumulative budget and must either refuse service or fall back to the cheapest model for the entire remaining tail—degrading quality precisely when demand (and revenue opportunity) is highest. The rate constraint thus inverts the relationship between traffic variability and operational risk: volume spikes that would trigger a budget crisis under a knapsack formulation become straightforward revenue opportunities under a per-request rate target. This makes the rate-constraint formulation strictly preferable for open-ended production deployments where traffic volume is variable and unpredictable.

Obtaining Discriminative Rewards in Practice. Our experiments use an automated LLM judge that produces continuous, multi-dimensional scores—an ideal signal for $K \geq 3$ routing, where the bandit must distinguish models whose average quality differs by only a few percentage points (e.g., 0.908 vs. 0.921 in our portfolio).

In the specific context of LLM routing, binary feedback (thumbs-up/down) is insufficient for $K \geq 3$ portfolios. The root cause is that modern LLMs at adjacent price tiers produce acceptable responses on the majority of prompts; a binary threshold therefore clusters frontier and mid-tier models into a single “strong” category, collapsing the multi-model problem to a $K=2$ strong-vs.-weak triage. We observe this directly in our portfolio: a natural binarisation threshold ($\tau=0.85$) causes the two stronger models to tie on 85.8% of prompts, eliminating the router’s ability to find quality within a budget.

This structural limitation is widely reflected in the LLM routing literature. RouteLLM [28] and BaRP [40] both frame routing as an explicit binary strong/weak decision trained on pairwise preference data; RouteLLM acknowledges this $K=2$ restriction as a limitation and identifies extension to multiple models as future work. Recent benchmarking studies reinforce the point: LLMRouter-Bench [20] identifies oversimplified binary accuracy metrics and imbalanced model pools—where a single dominant model masks inter-tier distinctions—as key evaluation failures for multi-model routing. Conversely, every production-oriented router that supports $K > 2$ portfolios adopts continuous quality signals: MixLLM [41] uses predicted accuracy scores across $K=5$ models, and PROTEUS [5] optimises continuous SLA targets across $K=11$ –14 models via Lagrangian RL. We emphasise that this observation is specific to LLM routing, where inter-model quality gaps are narrow and most prompts are “easy” for multiple tiers; we do not claim it generalises to other bandit domains where binary rewards may carry larger inter-arm separation.

In customer-facing deployments where a dedicated LLM judge is too costly or too slow, four practical strategies can produce the continuous reward signal that multi-model routing requires:

- (1) **Asynchronous LLM judge (sampled).** Rather than evaluating every response, teams can judge a random sample (e.g., 5–10%) asynchronously and use the resulting scores as stochastic rewards. LinUCB converges under partial feedback; the effective sample rate simply slows learning proportionally—acceptable when warmup priors already provide a strong initialisation.
- (2) **Multi-criteria human ratings.** Replacing a binary thumbs-up/down with a short Likert scale (e.g., 1–5 stars) or a two-question rubric (“Was this helpful?” + “Was this complete?”) produces a $[0, 1]$ composite that preserves inter-model gaps. Even a three-point scale (poor/acceptable/excellent) retains sufficient discrimination for $K=3$ –5 portfolios.
- (3) **Implicit behavioural composites.** Many applications already log signals that correlate with response quality: whether the user copied the output, the time spent reading it, follow-up question rate, code acceptance or edit distance, task completion flags, or retry/regeneration rate. Individually each signal is noisy and near-binary, but combining m such indicators into a normalised composite $r_t = \frac{1}{m} \sum_i s_i \in [0, 1]$ yields a continuous reward whose variance decreases as $1/m$. This approach requires no explicit rating UI and is compatible with high-volume, low-touch applications.
- (4) **Windowed binary aggregation.** If only binary feedback is available, the per-arm success proportion over a rolling window (e.g., last 200 observations per arm) approximates a continuous expected reward. However, this strategy requires substantially more traffic per arm before inter-model gaps become statistically detectable, making it viable only for high-volume deployments.

ParetoBandit’s reward interface accepts any scalar $r_t \in [0, 1]$, so all four strategies—and hybrid combinations—are drop-in compatible without modifying the routing algorithm.

Delayed Feedback and State Persistence. Our experiments assume immediate feedback, but three common production scenarios create a gap between routing and reward observation. First, *automated judge evaluation* (as used in this paper) is itself an LLM call that adds latency and cost; production systems typically batch these evaluations asynchronously rather than blocking the response path. Second, *human feedback* (thumbs-up/down, post-edit acceptance, task-completion signals) is inherently delayed—users do not rate responses immediately, and downstream quality signals such as conversion or error rates may take hours or days to materialize. Third, in microservice architectures the *routing service and feedback-ingestion service may be separate processes* or even separate machines, so the context vector x_t used at route time is no longer in memory when the reward r_t arrives.

LinUCB’s update rule requires the *exact* context vector that was used at selection time; re-encoding the prompt is not reliable because the embedding model version, PCA artifact, or whitening scales may have changed between routing and feedback ingestion. The context vector must therefore be persisted at route time and retrieved by request ID when feedback arrives. Without such persistence, a process restart (deployment, crash, autoscaling event) also discards all learned state, forcing the router to cold-start from priors and re-learn from scratch. Existing bandit-based routers [22, 31, 40]

maintain state only in process memory, making them incompatible with all three scenarios above.

ParetoBandit addresses this with a pluggable ContextStore abstraction backed by two built-in implementations:

- **EphemeralContextStore:** A bounded in-memory deque for low-latency deployments where evaluation is synchronous (e.g., automated metrics computed inline).
- **SQLiteContextStore:** A disk-persisted store using WAL-mode SQLite with TTL-based expiration, supporting millions of entries, concurrent reads/writes ($>10K$ writes/sec), and week-scale retention for RLHF workflows—with zero external dependencies. A separate CheckpointManager periodically snapshots the learned bandit state ($A_a, A_a^{-1}, b_a, \lambda_t$) via atomic writes, enabling crash recovery with automatic registry-drift handling when the model portfolio changes between restarts. The storage layer is extensible: implementing the ContextStore interface for Redis or S3 requires overriding three methods without modifying router logic.

Embedding Generation Costs. Contextual bandits require a feature vector x_t for every prompt. Generating this vector using a dense embedding model (e.g., SentenceTransformers) incurs a small latency and compute cost. While negligible compared to the inference cost of frontier LLMs, this overhead must be factored into the overall budget B for ultra-low-cost, high-throughput deployments. Utilizing smaller, faster embedding models or distilling features directly from the prompt text are practical mitigations.

Linearity of the Reward Model. LinUCB assumes that the expected reward is a linear function of the context: $\mathbb{E}[r_t | x_t, a] = \theta_a^\top x_t$. This is a strong modelling assumption. LLM reward functions are likely nonlinear in prompt semantics—prompt difficulty, model-specific failure modes, and interaction effects between semantic dimensions do not naturally compose linearly in embedding space. ParetoBandit partially mitigates this through two mechanisms: (i) the disjoint formulation gives each arm its own θ_a , which is strictly more expressive than a single shared linear model, and (ii) the PCA-whitened embedding pipeline (Section 5) projects prompts into the subspace where the leading principal components concentrate the routing-relevant signal, improving the linear approximation in practice. Empirically, this combination is sufficient to produce consistent Pareto gains across all experiments in this paper.

Nevertheless, richer reward surfaces could potentially be captured by *neural contextual bandits*. NeuralUCB [45] replaces the linear reward model with a deep network while constructing UCB-style confidence bounds via the neural tangent kernel, achieving $\tilde{O}(\sqrt{T})$ regret without structural assumptions on the reward function. NeuralTS [42] provides an analogous extension for Thompson Sampling. Adopting either approach for LLM routing is appealing in principle but introduces three open challenges that the linear formulation avoids:

- (1) *Non-stationary adaptation.* LinUCB’s geometric forgetting reduces to a scalar multiply on the sufficient statistics (A_a, b_a); neural networks have no analogous closed-form decay and would require more complex continual-learning or replay-based forgetting strategies to handle reward drift.
- (2) *Warm-start initialization.* Seeding LinUCB from offline data is a single matrix addition; transferring offline knowledge into

a neural reward model requires careful pre-training or meta-learning to avoid catastrophic forgetting of the prior during online fine-tuning.

- (3) *Latency.* Our end-to-end decomposition (Appendix A.1) shows that LinUCB’s routing decision (22 μ s) contributes < 1% of the full critical-path latency (~8 ms on CPU); prompt embedding dominates at 97%. This cost structure means a neural reward model would add its forward pass *on top of* the shared embedding cost—the vLLM Semantic Router [23] reports 9–14 ms per GPU-accelerated classifier, and that figure *is* the combined embed-and-classify cost. ParetoBandit separates these concerns: the embedding is a shared infrastructure cost that every contextual router pays (RouteLLM, CSCR, LLM Bandit alike), while the routing math itself is effectively free. Replacing that 22 μ s decision with a neural forward-and-backward pass would increase routing-specific overhead by orders of magnitude without reducing the embedding bottleneck. Relative to the LLM inference call itself, the full ~8 ms routing pipeline adds < 1% overhead even to the fastest production model (Claude Haiku 4.5 at 952 ms [15]), and < 0.2% for typical workloads—the routing decision is invisible to the end user. No peer-reviewed LLM router reports per-decision or end-to-end latency [5, 22, 28, 40, 41], making this decomposition—and the finding that routing math is negligible—a contribution in its own right.

Investigating whether the empirical gains from nonlinear reward models justify these engineering costs—and designing forgetting and warm-start mechanisms that work without closed-form sufficient statistics—is a compelling direction for future work.

Cross-Model Parameter Sharing. ParetoBandit uses disjoint LinUCB: each arm maintains fully independent sufficient statistics (A_a, b_a), so observations from one model contribute nothing to the quality estimates of another. This independence is conservative—it avoids negative transfer between dissimilar models—but it also means that the router cannot exploit the behavioural similarities that LLMs frequently exhibit. Models from different providers that are trained on overlapping data, fine-tuned with similar RLHF procedures, or positioned at the same price tier often share quality patterns: they tend to succeed on the same prompt types (straightforward instruction-following, factual retrieval) and struggle on the same failure modes (adversarial prompts, multi-step reasoning under ambiguity).

A promising future direction is a *hybrid* reward model that decomposes per-arm rewards into a shared component learned across all models and an arm-specific residual—the hybrid LinUCB formulation of Li et al. [21], where $r_t = z_t^\top \beta + x_t^\top \theta_a$. Three benefits connect directly to the challenges studied in this paper:

- (1) *Sample-efficient exploration under budget constraints.* Under tight budgets, expensive arms receive very few observations (e.g., < 1% Gemini traffic at the tight target). With shared parameters, every observation—including those from cheap, frequently selected arms—updates the shared component β , indirectly improving quality estimates for under-explored arms without spending additional budget.

- (2) *Faster adaptation under non-stationarity.* When a provider update affects multiple models similarly (e.g., a shared inference-infrastructure change, or a training-data refresh that shifts quality on a broad prompt category), the shared component would propagate the signal across arms immediately, rather than requiring each arm to independently detect the drift.
- (3) *Accelerated cold-start onboarding.* The model onboarding experiment (Section 6.5) showed that a cold-started arm requires ~233 steps to reach sustained adoption. With shared parameters, a newly registered model would immediately inherit the shared component’s knowledge of how prompt semantics relate to model quality, potentially reducing this discovery phase substantially.

The key design challenge is calibrating the shared-vs.-specific split: too much sharing risks negative transfer when models are genuinely dissimilar (e.g., a code-specialist vs. a general-purpose model), while too little sharing collapses back to the disjoint case. Investigating this tradeoff—particularly under the budget and non-stationarity pressures that define production LLM routing—is a natural next step.

8 Conclusion

We introduced ParetoBandit, an open-source framework for budget-constrained, non-stationary LLM routing. By enforcing budget constraints through an online Primal-Dual Lagrangian relaxation and integrating geometric forgetting, we provide a robust solution to the dynamic challenges of production deployments. Our system demonstrates precise budget compliance (0.97–1.02 $\times$ target), eliminates the need for manual hyperparameter tuning of static cost penalties, and autonomously adapts to silent model updates and mid-cycle price drops.

Quantified operational impact. The experiments in this paper yield four concrete, verifiable claims that distinguish ParetoBandit from static routing approaches:

- (1) *Budget compliance.* In Phase 1 across all tested budget regimes, ParetoBandit maintains spend within 2% of the dollar target (0.97 $\times$ –1.00 $\times$); after the price drop, it correctly relaxes when the constraint becomes slack rather than artificially inflating spend. Static penalties deviate by up to 2.74 $\times$, translating—at scale—to either SLA-violating overruns or wasted quality headroom. This precision enables dollar-denominated SLAs (e.g., “average spend $\leq 1.05 \times B$ over any rolling window”) that static penalties structurally cannot guarantee.
- (2) *Autonomous adaptation.* Under a mid-cycle price drop, ParetoBandit re-routes 72% of traffic to the newly cost-effective frontier model within hundreds of requests, capturing a +0.071 quality lift without manual intervention. Static systems require an offline recalibration cycle—retraining, grid search, redeployment—to exploit the same opportunity.
- (3) *Deployment reliability.* Under catastrophic model failure, ParetoBandit maintains budget compliance (0.97–1.03 $\times$ target) across all nine phase-budget conditions, while the Naive Bandit’s static penalty produces budget violations of up to 7.58 $\times$ (Section 6.3). The dual-variable mechanism provides closed-loop cost control through severe disruptions—online learning alone is insufficient.

- (4) *Cold-start model onboarding*. A single `register_model()` call—with no offline evaluation or warmup priors—suffices to integrate a new model into a budget-constrained router. Across 80 trials (Section 6.5; Appendix A.5), LinUCB’s exploration bonus discovers and sustains adoption of the cold-started arm within ~233 steps, displacing the economically rational incumbent, while the BudgetPacer maintains compliance without reconfiguration.

These claims are derived from offline simulation and should be validated through online A/B testing before being projected to specific dollar savings at production scale. Nevertheless, the magnitude of the compliance and adaptation gaps—162% budget overshoot for static baselines versus $\leq 2\%$ for ParetoBandit—provides strong evidence that the operational benefits transfer to real deployments.

Beyond the algorithmic contributions, the library provides the production scaffolding that enterprise deployment demands—pluggable encoders and PCA artifacts for domain-specific embeddings, persistent context stores for delayed human feedback, atomic checkpointing for crash recovery, and a single-call `register_model()` API that onboards new models at runtime from a cold start without offline evaluation or budget reconfiguration. We release the full codebase, experiment scripts, and pre-trained artifacts to enable both immediate enterprise deployment and reproducible research on non-stationary LLM routing.

References

- [1] Claudio Acquaviva. 2025. AI Gateway Benchmark: Kong AI Gateway, Portkey, and LiteLLM. <https://konghq.com/blog/engineering/ai-gateway-benchmark-kong-ai-gateway-portkey-litellm>. Proxy-only benchmark on c5.4xlarge EKS nodes (16 vCPU). Kong p95 = 24 ms, Portkey p95 $\approx$ 70 ms; WireMock baseline p95 = 24 ms..
- [2] Shipra Agrawal and Nikhil R Devanur. 2014. Bandits with Concave Rewards and Convex Knapsacks. In *Proceedings of the Fifteenth ACM Conference on Economics and Computation (EC)*. ACM, 989–1006.
- [3] Melissa Ailem, Katerina Marazopoulou, Charlotte Siska, and James Bono. 2024. Examining the Robustness of LLM Evaluation to the Distributional Assumptions of Benchmarks. *arXiv preprint arXiv:2404.16966* (2024).
- [4] Dor Amir. 2026. NadirClaw: Open-Source LLM Router. <https://github.com/doramirdor/NadirClaw>. Sentence-embedding-based tier router; 8–12 ms E2E classification overhead reported..
- [5] Amit Singh Bhatti, Vishal Vaddina, and Dagnachew Birru. 2026. PROTEUS: SLA-Aware Routing via Lagrangian RL for Multi-LLM Serving Systems. *arXiv preprint arXiv:2601.19402* (2026).
- [6] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. In *Proceedings of the 40th International Conference on Machine Learning*.
- [7] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. How Is ChatGPT’s Behavior Changing over Time? *arXiv preprint arXiv:2307.09009* (2023).
- [8] Shuhao Chen, Weisen Jiang, Baijiong Li, Jianping Gao, and Caihua Zhang. 2024. RouterDC: Query-Based Router by Dual Contrastive Learning for Assembling Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [9] Yuanlong Chen. 2025. A Practical Guide to Budget Pacing Algorithms in Digital Advertising. *arXiv preprint arXiv:2503.06942* (2025).
- [10] Y. Chen and D. Lee. 2025. Monitoring Black-Box LLMs for Silent Updates and Model Drift. *arXiv preprint arXiv:2504.12335* (2025).
- [11] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948* (2025).
- [12] Dujan Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V.S. Levi, and Ahmed Hassan Awadallah. 2024. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. In *International Conference on Learning Representations (ICLR)*.
- [13] Tao Feng, Yanzhen Shen, and Jiaxuan You. 2025. GraphRouter: A Graph-based Router for LLM Selections. In *International Conference on Learning Representations (ICLR)*.
- [14] João Gama, Indrè Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A Survey on Concept Drift Adaptation. *Comput. Surveys* 46, 4 (2014), 1–37.
- [15] Kunal Ganglani. 2026. LLM API Latency Benchmarks [2026]: 5 Models Compared. <https://kunalganglani.com/blog/llm-api-latency-benchmarks-2026>. Independent benchmark of five production LLM APIs (GPT-4.1, GPT-4.1 Mini, Gemini 2.5 Flash, Claude Haiku 4.5, Claude Sonnet 4). Three prompt sizes, streaming TTFT, total latency, and throughput..
- [16] Aurélien Garivier and Eric Moulines. 2011. On Upper-Confidence Bound Policies for Switching Bandit Problems. In *Proceedings of the 22nd International Conference on Algorithmic Learning Theory (ALT)*. Springer, 174–188.
- [17] Google. 2024. Gemini 1.5 Flash Price Reduction. <https://ai.google.dev/gemini-api/docs/pricing>. Input token price cut from \$0.35 to \$0.075 per million tokens (~78% reduction); output from \$1.05 to \$0.30 (~71% reduction). Effective August 12, 2024..
- [18] Branislav Kveton, Martin Mladenov, Chih-Wei Hsu, Manzil Zaheer, Csaba Szepesvári, and Craig Boutilier. 2023. Cutting to the Chase with Warm-Start Contextual Bandits. *Knowledge and Information Systems* 65, 4 (2023), 1611–1644.
- [19] Tor Lattimore and Csaba Szepesvári. 2020. *Bandit Algorithms*. Cambridge University Press.
- [20] Hao Li, Yiqun Zhang, Zhaoyan Guo, Chenxu Wang, Shengji Tang, Qiaosheng Zhang, Yang Chen, Biqing Qi, Peng Ye, Lei Bai, Zhen Wang, and Shuyue Hu. 2026. LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing. *arXiv preprint arXiv:2601.07206* (2026).
- [21] Lihong Li, Wei Chu, John Langford, and Robert E Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web*. 661–670.
- [22] Yang Li. 2025. LLM Bandit: Cost-Efficient LLM Generation via Preference-Conditioned Dynamic Routing. *arXiv preprint arXiv:2502.02743* (2025).
- [23] Xunzhuo Liu, Bowei He, Xue Liu, Andy Luo, Haichen Zhang, and Huamin Chen. 2026. 98 $\times$ Faster LLM Routing Without a Dedicated GPU: Flash Attention, Prompt Compression, and Near-Streaming for the vLLM Semantic Router. *arXiv preprint arXiv:2603.12646* (2026).
- [24] Wanqin Ma, Chenyang Yang, and Christian Kästner. 2024. (Why) Is My Prompt Getting Worse? Rethinking Regression Testing for Evolving LLM APIs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering—Software Engineering for AI (CAIN)*.
- [25] Zitian Ma et al. 2025. OmniRouter: Budget and Performance Controllable Multi-LLM Routing. *arXiv preprint arXiv:2502.20576* (2025).
- [26] Menlo Ventures. 2025. 2025 Mid-Year LLM Market Update: Foundation Model Landscape + Economics. <https://menlovc.com/perspective/2025-mid-year-llm-market-update>. Survey of 150+ technical leaders. Enterprise LLM API spend reached \$8.4B by mid-2025, more than doubling from \$3.5B in late 2024..
- [27] Microsoft. 2025. Azure OpenAI Service Statistics and Enterprise Adoption. <https://azure.microsoft.com/en-us/products/ai-services/openai-service>. 230,000 organizational deployments; 100+ trillion tokens processed quarterly; 85% of Fortune 500 companies using Microsoft AI..
- [28] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *International Conference on Learning Representations (ICLR)*.
- [29] OpenAI. 2025. GPT-4 Turbo with Vision: Price Drop Announcement. <https://openai.com/index/new-models-and-developer-products-announced-at-devday>. Input token price reduced from \$0.03 to \$0.01 per 1K tokens (67% cut); output from \$0.06 to \$0.03 (50% cut). Step change effective immediately..
- [30] Haochen Pan, Guy Tennenholtz, Shie Mannor, Chih-Wei Chi, Rob Brekelmans, Pratik Shah, and Ambuj Tewari. 2024. Jump Starting Bandits with LLM-Generated Prior Knowledge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 19858–19873.
- [31] Pranoy Panda, Raghav Magazine, Chaitanya Devaguptapu, Sho Takemori, and Vishal Sharma. 2025. Adaptive LLM Routing under Budget Constraints. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. arXiv:2508.21141.
- [32] Arian Pasquali. 2026. Intelligent LLM Routing: How Auto Router Cuts Costs by 25–70% While Preserving Quality. <https://router.org.ai/blog/auto-router-intelligent-llm-routing>. Commercial router; <40 ms routing overhead per request..
- [33] Portkey AI. 2025. The Most Reliable AI Gateway for Production Systems. <https://portkey.ai/blog/the-most-reliable-ai-gateway-for-production-systems>. Sub-millisecond hot-path latency; 122 KB gateway footprint; >10B requests/month in production..
- [34] Cheng Qian, Zuxin Liu, Shirley Kokane, Akshara Prabhakar, Jieli Qiu, Haolin Chen, Zhiwei Liu, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. 2025. xRouter: Training Cost-Aware LLMs Orchestration System via Reinforcement Learning. *arXiv preprint arXiv:2510.08439* (2025).
- [35] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 3982–3992.
- [36] Reza Shirkavand, Shangqian Gao, Peiran Yu, and Heng Huang. 2025. Cost-Aware Contrastive Routing for LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- [37] Zijian Sun, Hanling Xue, Jie Wang, and Zhi-Xuan Tu. 2025. Lookahead Routing for Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [38] Alan D. Thompson. 2026. Timeline of AI and Language Models. <https://lifearchitect.ai/timeline/>. Accessed: Feb 2026. Curated timeline of major frontier LLM releases from major providers, 2017–2026..
- [39] Pat Verga, Sebastian Hofstätter, Sophia Althammer, Yixuan Su, Aleksandra Gupta, Hossein Faez, Fabio Petroni, and Sebastian Riedel. 2024. Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models. *arXiv preprint arXiv:2404.18796* (2024).
- [40] Wei Wang, Tiankai Yang, Hongjie Chen, Yue Zhao, Franck Dernoncourt, Ryan A Rossi, and Hoda Eldardiry. 2025. Learning to Route LLMs from Bandit Feedback: One Policy, Many Trade-offs. *arXiv preprint arXiv:2510.07429* (2025).
- [41] Xinyuan Wang, Yanchi Liu, Wei Cheng, Xujiang Zhao, Zhengzhang Chen, Wen-chao Yu, Yanjie Fu, and Haifeng Chen. 2025. MixLLM: Dynamic Routing in Mixed Large Language Models. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 10912–10922.
- [42] Weitong Zhang, Dongruo Zhou, Lihong Li, and Quanquan Gu. 2021. Neural Thompson Sampling. In *International Conference on Learning Representations (ICLR)*.
- [43] Jitian Zhao, Changho Shin, Tzu-Heng Huang, Satya Sai Srinath Namburi, and Frederic Sala. 2025. CARE: Confounder-Aware Aggregation for Reliable LLM Evaluation. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [44] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P Xing, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685* (2023).
- [45] Dongruo Zhou, Lihong Li, and Quanquan Gu. 2020. Neural Contextual Bandits with UCB-based Exploration. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. PMLR, 11492–11502.

A Supplementary Experiments

A.1 Routing Latency Microbenchmark (Table 6, Figure 7)

We profile ParetoBandit’s per-request routing overhead to validate the computational-efficiency claims in Section 4.4. Four configurations isolate the individual contributions of *PCA dimensionality reduction* ($d=26$ vs. $d=385$) and *Sherman-Morrison incremental updates* (cached A^{-1} vs. full `np.linalg.inv` on every route call). All configurations use $K=3$ arms, synthetic whitened context vectors, and 4,500 measured route + update cycles (500-round warmup excluded).

Table 6: Per-request routing latency (μ s) and throughput. ParetoBandit’s production configuration (top row) adds $< 39 \mu$ s per request—negligible relative to LLM inference times of 100–2,000 ms.

Configuration	Route (μ s)		Update (μ s)		Throughput (req/s)
	p50	p95	p50	p95	
ParetoBandit ($d=26$)	21.7	25.5	16.8	19.5	24,347
ParetoBandit ($d=385$)	152.8	214.4	367.8	427.8	1,885
Naive Inv. ($d=26$)	49.2	51.2	4.0	4.7	18,708
Naive Inv. ($d=385$)	4,572	4,902	128.1	702.6	202

Why ParetoBandit’s route is fast but its update is slower than naive. The most striking pattern in Table 6 is that ParetoBandit’s *update* is slower than the naive baseline (17μ s vs. 4μ s at $d=26$), yet its *route* is $2.3\times$ faster—and at $d=385$ the route gap widens to $30\times$. This is an intentional architectural trade-off: ParetoBandit *amortizes* the cost of maintaining the matrix inverse into the update path so the latency-critical route path is minimal.

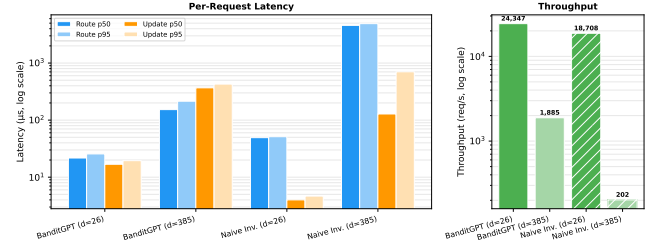


Figure 7: Per-request latency (left, log scale) and throughput (right, log scale) for four router configurations. The combined effect of PCA reduction ($385 \rightarrow 26$ dimensions) and Sherman-Morrison caching yields a $120\times$ throughput improvement over the naive baseline at raw dimension.

- **Naive baseline.** Update performs only the $O(d^2)$ outer-product addition $A_a += x_t x_t^\top$ (4μ s at $d=26$). Route must then recompute A_a^{-1} from scratch via $O(d^3)$ inversion for each of K arms before computing the UCB score. At $d=385$ this costs 4.6 ms per route call.
- **ParetoBandit.** Update pays a higher constant: the $O(d^2)$ Sherman-Morrison rank-1 correction on A_a^{-1} (Eq. 9), plus a cached $\hat{\theta}_a = A_a^{-1} b_a$ refresh. Route then needs only one $O(d)$ dot product ($\hat{\theta}_a^\top x_t$) and one $O(d^2)$ quadratic form ($x_t^\top A_a^{-1} x_t$) per arm—no inversion at all.

Because routing sits on the user-facing critical path while updates can be processed asynchronously (Section 7), this trade-off is strongly favorable: it shifts work from the latency-sensitive path to the throughput-tolerant path.

Key findings.

- (1) **Production overhead is negligible.** At the production dimension ($d=26$), ParetoBandit completes a full route + update cycle in 39μ s (p50), adding $< 0.004\%$ latency to even the fastest LLM call (~ 100 ms time-to-first-token for Llama-3.1-8B). The router sustains $> 24,000$ req/s on a single CPU core—sufficient for all but the most extreme throughput tiers.
- (2) **Sherman-Morrison accelerates selection by $30\times$ at high dimension.** At $d=385$, the naive baseline spends 4.6 ms per route call computing $K=3$ full matrix inversions. ParetoBandit’s cached inverse reduces this to 153μ s—a $30\times$ speedup—because selection requires only a matrix-vector product ($O(d^2)$) rather than an inversion ($O(d^3)$). At $d=26$ the gap narrows to $2.3\times$ because $O(d^3)$ with a small constant is already fast.
- (3) **PCA gives an additional $13\times$ throughput gain.** Within the Sherman-Morrison configurations, reducing d from 385 to 26 improves throughput from 1,885 to 24,347 req/s ($12.9\times$). This is consistent with the expected $O(d^2)$ scaling: $(385/26)^2 \approx 219$, with overhead constants (locking, Python dispatch) compressing the realized ratio.
- (4) **Combined improvement: $120\times$.** The full production stack (PCA + Sherman-Morrison at $d=26$) achieves $120\times$ higher throughput than the naive baseline at raw embedding dimension (24,347 vs. 202 req/s), driven by three mutually reinforcing design choices: PCA reduces the matrix size ($O(d^2)$ scaling), Sherman-Morrison eliminates per-route inversion ($O(d^2)$ vs. $O(d^3)$), and cached

$\hat{\theta}_a$ reduces the expected-reward computation to a single $O(d)$ dot product per arm.

- (5) **Comparison with published routing systems.** Table 7 contextualises ParetoBandit’s overhead against published latencies from LLM routing systems, AI gateways, and infrastructure baselines. Two comparisons are relevant: (a) the *routing-decision* overhead, which isolates the model-selection computation from the shared embedding cost that all contextual routers pay, and (b) the *end-to-end critical-path* latency a user experiences before the LLM call begins.

Routing-decision overhead. ParetoBandit’s `route()` completes in 22 μ s (p50) on a single CPU core. No other LLM routing system reports a comparable figure. Among bandit-based LLM routers—LLM Bandit [22], BaRP [40], and MixLLM [41]—none publish per-decision latency, though all emphasise that routing decisions should be “lightweight” [22]. Classifier-based routers such as RouteLLM [28] (BERT, causal LLM, or matrix-factorization classifiers) also do not report routing-decision latency. CSCR [36] claims “microsecond” k -NN lookups via a FAISS index, but this figure excludes the contrastive encoder forward pass required to produce the query embedding, so the true end-to-end decision latency is substantially higher.

The best published end-to-end routing latency comes from the vLLM Semantic Router [23], a GPU-accelerated system-level router that uses ModernBERT classifiers on an AMD MI300X for domain classification, safety filtering, and PII detection.¹ It reports 9–14 ms per classifier for short prompts (≤ 512 tokens) and 22 ms end-to-end with three concurrent classifiers. ParetoBandit’s 22 μ s routing decision is thus $\sim 400\times$ faster than the neural classification step alone.

End-to-end critical path (embedding included). In production, ParetoBandit’s critical path includes prompt embedding (MiniLM-L6-v2) and PCA projection before the `route()` call. We benchmark this full pipeline on CPU (Apple M-series, single-threaded, 100 iterations, 3-pass warmup):

Stage	p50	p95	% of total
Embed (MiniLM-L6-v2, CPU)	8.1 ms	9.0 ms	97.4%
PCA + whitening	0.15 ms	0.17 ms	1.8%
<code>route()</code>	0.058 ms	0.069 ms	0.7%
Total E2E (CPU)	8.3 ms	9.3 ms	100%

The embedding step dominates at 97% of total time; the routing decision contributes $< 1\%$. This cost structure has a direct implication: *the routing algorithm is not the bottleneck*—prompt embedding is, and it is a shared infrastructure expense that every contextual router must pay. RouteLLM, CSCR, LLM Bandit, and any other embedding-dependent router face the same floor. The vLLM Semantic Router avoids a separate embedding step by folding it into the ModernBERT classification forward pass, but at a higher total cost (9–22 ms on GPU vs. 8.3 ms on CPU for ParetoBandit).

¹The vLLM Semantic Router performs content classification for backend selection rather than quality-cost optimisation; we include it because it provides the only published end-to-end neural routing latency on representative hardware.

Practitioner-reported latencies corroborate this picture. NadirClaw, an open-source embedding-based LLM router [4], reports 8–12 ms E2E using sentence embeddings for tier classification—the same range as ParetoBandit, confirming that the embedding cost dominates for any system built on this architecture. The commercial Orq.ai Auto Router reports < 40 ms [32]—higher overhead, consistent with a heavier classification pipeline.

No peer-reviewed LLM router—bandit-based, classifier-based, or RL-based—reports per-decision or end-to-end routing latency (Table 7). This decomposition, and the finding that the routing decision contributes $< 1\%$ of total overhead, is to our knowledge the first such analysis in the LLM routing literature.

With a GPU-hosted embedding model (~ 0.3 – 0.5 ms for MiniLM-L6-v2), the total E2E critical path would drop to ~ 0.5 – 1 ms—an order of magnitude below both neural classification alternatives and the embedding-bound CPU floor.

Table 7: Routing latency comparison across system types. *Decision only isolates the model-selection computation; E2E includes prompt encoding.* † System-level router (domain/safety classification for backend selection, not quality-cost optimisation). ‡ Reports only the k -NN index lookup, excluding the encoder forward pass. § Practitioner-reported (not peer-reviewed). “—” indicates latency was not reported.

System	Category	Scope	Latency
<i>Bandit-based LLM routers</i>			
ParetoBandit	LinUCB + Sherman-Morrison	decision only	22 μ s
ParetoBandit	+ MiniLM-L6 + PCA	E2E (CPU)	8.3 ms
LLM Bandit [22]	contextual MAB	—	—
BaRP [40]	contextual dueling bandits	—	—
MixLLM [41]	contextual bandits + MLP	—	—
<i>Classifier / embedding-based LLM routers</i>			
RouteLLM [28]	BERT / MF / causal LLM	E2E	—
CSCR [‡] [36]	k -NN via FAISS	lookup only	μ s (claimed)
HybridLLM [12]	BERT classifier	E2E	—
PROTEUS [5]	Lagrangian RL	E2E	—
OmniRouter [25]	constrained optimisation	E2E	—
Lookahead [37]	causal/masked LM predictor	E2E	—
<i>Practitioner LLM routers[§] (not peer-reviewed)</i>			
NadirClaw [§] [4]	sentence embed + classifier	E2E	8–12 ms
Orq.ai Auto Router [§] [32]	undisclosed (commercial)	E2E	< 40 ms
<i>Neural system-level routers (not quality-cost LLM routers)</i>			
vLLM SR [†] (1 clf.) [23]	ModernBERT + FlashAttn	E2E	9–14 ms
vLLM SR [†] (3 clf.) [23]	ModernBERT + FlashAttn	E2E	22 ms
vLLM SR [†] (8K tok.) [23]	ModernBERT + compression	E2E	50 ms
<i>Infrastructure baselines (no model selection)</i>			
Portkey Gateway [33]	L7 proxy	proxy only	< 1 ms
Kong AI Gateway [1]	L7 proxy	proxy only	24 ms (p95)

Routing overhead relative to LLM inference. The practical significance of routing latency depends on its magnitude relative to the LLM inference call it precedes. Table 8 contextualises ParetoBandit’s overhead against independently benchmarked production LLM API latencies [15].

Table 8: ParetoBandit’s routing overhead as a fraction of LLM inference latency across representative models and prompt lengths [15]. TTFT = time to first token (streaming); Total = full response latency. Routing overhead is the 8.3 ms end-to-end critical path (embedding + PCA + route()) on CPU.

LLM	Prompt	TTFT (ms)	Total (ms)	Routing / Total
Claude Haiku 4.5	short	639	952	0.87%
Claude Haiku 4.5	medium	597	3,954	0.21%
Gemini 2.5 Flash	medium	730	1,729	0.48%
GPT-4.1	short	889	1,770	0.47%
Claude Sonnet 4	long	1,216	20,445	0.04%
GPT-4.1 Mini	long	2,501	18,075	0.05%

Even against the fastest production model and shortest prompt (Claude Haiku 4.5, 952 ms total latency), ParetoBandit’s full end-to-end routing pipeline adds < 1% overhead. For typical workloads (2–5 s total latency), routing contributes 0.2–0.4%; for long-form generation (7–20 s), the overhead drops below 0.1%. The route() decision alone (22 μ s) is < 0.004% of any LLM call—effectively unmeasurable in production traces. In practical terms, **the routing decision is invisible to the end user**: the time spent selecting the optimal model is orders of magnitude smaller than the time spent waiting for that model to respond.

A.2 Cold-Start vs Warmup Priors (Figure 8)

We isolate the contribution of warmup priors by comparing ParetoBandit with offline priors against a tabula-rasa cold start (identity covariance, zero bias) on the $K=3$ stationary portfolio under three budget regimes: unconstrained (no pacer), tight ($B=\$3.0\times 10^{-4}$ /req), and moderate ($B=\$6.6\times 10^{-4}$ /req). Each condition uses the optimal hyperparameters from the Experiment 04 sweep: ParetoBandit uses $\alpha=0.1$, $n_{\text{eff}}=1,000$, $\gamma=0.995$; Tabula Rasa uses $\alpha=0.1$, $n_{\text{eff}}=1$, $\gamma=0.995$ (disjoint LinUCB). Budget-constrained conditions use the Primal-Dual BudgetPacer with the same learning rate and λ_{max} as Experiment 01. Following the standard contextual-bandit cumulative-regret protocol, the router learns and is evaluated simultaneously on the held-out test split ($n=1,824$, disjoint from warmup-prior training on train.jsonl and hyperparameter selection on val.jsonl).

Budget	Condition	Regret (95% CI)	R@200 (95% CI)	Rwd
None	Warmup	74.5 [73.2, 75.7]	6.6 [6.2, 7.1]	0.923
	Tabula Rasa	83.1 [81.4, 84.9]	12.5 [10.9, 14.1]	0.918
	Random	146.3 [144.2, 148.5]	16.1 [15.2, 17.1]	0.883
Tight	Warmup	170.8 [167.9, 173.8]	21.8 [20.0, 23.6]	0.870
	Tabula Rasa	179.9 [172.2, 187.6]	23.8 [21.7, 26.0]	0.865
Moderate	Warmup	151.4 [146.0, 156.8]	16.7 [14.8, 18.5]	0.880
	Tabula Rasa	151.7 [145.2, 158.2]	19.7 [17.3, 22.1]	0.880

Twenty seeds; 95% bootstrap CI; test split ($n=1,824$). p -values are from paired t -tests (warmup vs. tabula rasa, same seed).

Unconstrained. Without a budget pacer, warmup priors reduce total regret by 10% (74.5 vs. 83.1; $\Delta=-8.7$, $p < 10^{-5}$) and early regret (R@200) by 47% (6.6 vs. 12.5). The advantage is concentrated in the first ~200 steps (Figure 8), where the warmup router begins with

informed arm-quality estimates and avoids the exploratory phase that the cold-start router must perform. Beyond step ~400, the two routers converge, but the regret gap established early persists through the full horizon.

Tight budget. Under a tight budget the warmup advantage persists (170.8 vs. 179.9; $\Delta=-9.1$, $p=0.059$; 5% reduction). The budget pacer penalises expensive arms, so the cold-start router’s early UCB-driven exploration of Gemini-Pro (~29 $\times$ costlier than Mistral-Large: \$0.0151/req vs. \$0.0005/req in our dataset) consumes real budget without delivering proportionate information gain. The warmup router, already knowing each arm’s quality profile, allocates its limited budget more efficiently from step 1.

Moderate budget. At moderate budgets the warmup advantage vanishes (151.4 vs. 151.7; $\Delta=-0.3$, $p=0.94$, not significant). The budget constraint forces both routers toward similar affordable arm mixes: at the final step the warmup router allocates 59% Mistral-Large / 39% Llama-8B / 2% Gemini-Pro, and the tabula-rasa router allocates 67% / 31% / 2% (from logged arm fractions). Within this constrained decision space the routing problem is simple enough for the cold-start router to solve quickly, and the prior adds no incremental value.

Discussion. The budget-stratified results sharpen the recommendation: warmup priors matter most when the router has freedom to explore expensive arms (unconstrained, $p < 10^{-5}$); under tight budgets the effect persists ($p=0.059$), though the budget constraint attenuates the penalty of uninformed early exploration by limiting how much the cold-start router can spend on expensive arms. When the budget collapses the feasible arm set to a small, affordable subset (moderate), the prior’s knowledge about expensive arms becomes irrelevant.

Importantly, the warmup benefit is *transient by design*. The convergence beyond step ~400 visible in Figure 8 is a direct consequence of geometric forgetting ($\gamma=0.995$): after ~200 effective-memory steps, online evidence dominates and the initialisation is irrelevant. Appendix A.3 confirms this quantitatively: after a full validation burn-in (1,785 steps), warmup and tabula-rasa conditions achieve identical test regret (74.4 vs. 75.4, $p=0.29$) because the prior has decayed by $\gamma^{1785} \approx 10^{-4}$. Warmup priors are therefore best understood as a *deployment latency optimisation*—they eliminate the cold-start transient where uninformed exploration dominates, but the router’s steady-state quality is determined by online learning alone.

Business perspective. Intelligent LLM routing delivers simultaneous quality and cost gains because no single model dominates all queries. In our $K=3$ portfolio, oracle routing—selecting the per-query best model—achieves 3.3% higher quality than routing all traffic to the frontier model ((0.963 – 0.932)/0.932; mean reward on the test split) while reducing per-request cost by 66% ((1 – 0.005/0.015); mean oracle cost vs. mean Gemini-2.5-Pro cost on the test split, at March 2026 API pricing of \$1.25/M input and \$10.00/M output tokens²). This dual improvement—better quality *and* lower cost—arises because the frontier model is not universally best: in our

²Google Cloud Vertex AI pricing, <https://cloud.google.com/vertex-ai/generative-ai/pricing>, accessed March 2026.

test split, a cheaper model strictly outcores Gemini-2.5-Pro on 35% of prompts (and ties on a further 37%), so Gemini is the unique best choice on only 27% of queries. The direction of the result is consistent with RouteLLM, which shows that routers can maintain >95% of frontier quality at >50% lower cost in a $K=2$ setting [28].

To illustrate the scale at which these per-request differences compound, we project to enterprise traffic volumes. Azure OpenAI Service alone reports 230,000 organisational deployments processing over 100 trillion tokens per quarter [27]; a mid-sized deployment handling 10^5 – 10^6 queries/day is well within this range. The per-request cost gap between frontier-only and oracle routing in our portfolio is $\$0.0151 - \$0.0051 = \$0.0100/\text{req}$, so $10^5 \times \$0.010 \times 365 = \$365\text{K}/\text{year}$, scaling linearly to $\sim \$3.7\text{M}$ at 10^6 daily requests—with no quality sacrifice, since oracle routing *improves* quality over the single-model baseline.

Within this context, warmup priors protect routing quality during the critical—but transient—cold-start window. Under tight budgets, the 5% regret reduction ($(179.9 - 170.8)/179.9$; $p=0.059$) represents 5% of the router’s total quality–cost value recovered solely by starting with informed priors. Applying the same projection: $0.05 \times \$365\text{K} \approx \$18\text{K}/\text{year}$ at 10^5 daily requests, rising to $\sim \$183\text{K}$ at 10^6 . This captures the cost of cold-start exploration, where every uninformed call on a $\$0.015$ frontier arm under a tight budget simultaneously degrades quality *and* consumes budget that would otherwise fund later high-value routing decisions. The early regret reduction ($R@200$) is even starker: 47% in the unconstrained regime (6.6 [6.2, 7.1] vs. 12.5 [10.9, 14.1]) and 8% under a tight budget (21.8 [20.0, 23.6] vs. 23.8 [21.7, 26.0]). In production, these first few hundred queries correspond to the initial deployment window where user trust is most fragile—precisely when suboptimal routing is costliest.

These projections are illustrative: the per-request costs are specific to our three-model portfolio and March 2026 pricing, and the traffic volumes are drawn from public platform statistics rather than a single organisation’s logs. Actual savings depend on the specific model mix, prompt distribution, and pricing at deployment time. Nevertheless, warmup priors are easy to justify when calibrated on representative data: computing them from a labelled offline dataset takes minutes and requires no ongoing maintenance [18]. The key practical lever is n_{eff} , which controls how strongly the prior anchors the router. With well-calibrated priors (as in our experiments), $n_{\text{eff}}=1,000$ lets the router exploit prior knowledge immediately. When prior quality is uncertain—e.g., the offline dataset comes from a different prompt distribution or from stale model versions—a lower n_{eff} (e.g., 10–100) gives the router a gentle initial nudge while preserving enough exploration for rapid online correction. A dedicated prior-mismatch sensitivity analysis (Appendix A.4) quantifies these trade-offs across five quality levels—from well-calibrated through domain-restricted to actively inverted priors. The results show that *directionally correct* priors (including domain-mismatched priors trained exclusively on MMLU or GSM8K prompts) consistently reduce regret relative to a cold start at every n_{eff} tested ($p < 0.05$ for $n_{\text{eff}} \geq 100$), confirming that partial domain coverage does not harm the router. However, *systematically wrong* priors—where the best and worst models are inverted—cause measurable harm that *scales with n_{eff}* : at $n_{\text{eff}}=10$ the damage is

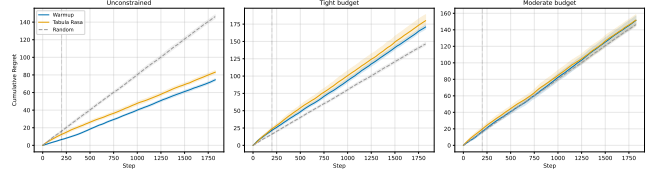


Figure 8: Cumulative regret: warmup priors vs. cold start across three budget regimes ($K=3$ stationary, 20 seeds, 95% bootstrap CI, held-out test split). Left: unconstrained—warmup reduces regret by 10% ($p < 10^{-5}$), concentrated in the first ~ 200 steps. Centre: tight budget—the advantage persists ($\Delta = -9.1$, $p=0.059$) as uninformed UCB exploration wastes scarce budget on expensive arms. Right: moderate budget—the gap vanishes ($p=0.94$) as the budget constraint collapses the feasible arm set, making the prior’s knowledge irrelevant.

statistically non-significant ($p=0.055$), but at $n_{\text{eff}}=1,000$ regret increases by 13% over cold start ($p < 10^{-6}$). The practical recommendation is therefore conditional: enable warmup priors whenever the prior is known to be directionally correct (i.e., the model-quality ranking it encodes is not systematically inverted), setting n_{eff} to reflect confidence in calibration quality—high when the offline data closely matches deployment traffic, low otherwise. If prior quality is entirely unknown, a conservative n_{eff} (10–100) limits worst-case exposure while still offering early-regret reduction on any non-adversarial prior (Appendix A.4, Figure 12). The computational cost of warmup priors is negligible, and the upside—at enterprise scale, where LLM API spend surpassed $\$8.4\text{B}$ by mid-2025 [26]—is economically meaningful.

Uncertainty evolution (Figure 9). To visualise the mechanism behind the early regret gap, Figure 9 plots $\text{tr}(A_a^{-1})$ —the total uncertainty per arm—for the unconstrained conditions across the trajectory. The context dimension is $d=26$ (PCA components) and the regularisation is $\lambda=1$ (default LinUCB). Cold-started arms (dashed lines) therefore begin at $\text{tr}(A_a^{-1}) = \text{tr}((\lambda I)^{-1}) = d/\lambda = 26$. Warmup-initialised arms (solid lines) start at $\text{tr}(A_a^{-1}) = 0.72$, reflecting the $n_{\text{eff}}=1,000$ pseudo-observations encoded in the offline prior—a $26/0.72 = 36\times$ uncertainty gap. Because the UCB exploration bonus scales as $\alpha\sqrt{x^\top A_a^{-1}x}$, this gap means the cold-start router must spend hundreds of steps exploring before it accumulates enough information to route as confidently as the warmup router does from step 1. With geometric forgetting ($\gamma=0.995$), arms that receive few observations see their uncertainty *grow* over time as the prior decays; the warmup Llama-8B trace rises from 0.72 to ~ 10 , illustrating how forgetting trades long-term certainty for adaptability. The two conditions largely converge by step ~ 400 for frequently selected arms (Gemini-Pro, Mistral-Large), consistent with the regret convergence in Figure 8.

A.3 Validation Burn-In Ablation (Figures 10–11)

The paper’s standard evaluation protocol runs the bandit through the validation split ($n=1,785$) before evaluating on the held-out test

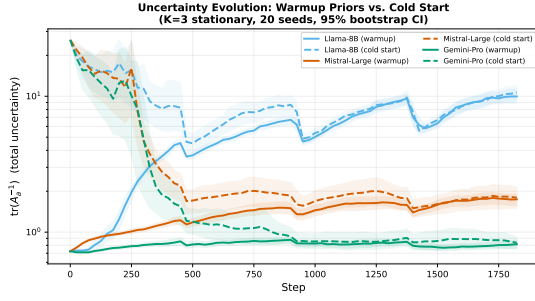


Figure 9: Uncertainty evolution for warmup vs. cold-start conditions (K=3 stationary, 20 seeds, 95% bootstrap CI). $\text{tr}(A_a^{-1})$ measures total per-arm uncertainty; higher values yield larger UCB exploration bonuses. Solid lines: warmup-initialised arms start at $\text{tr}(A_a^{-1}) \approx 0.72$, reflecting the $n_{\text{eff}}=1,000$ pseudo-observations encoded in the prior. Dashed lines: cold-started arms begin at $\text{tr}(A_a^{-1}) = d/\lambda = 26$ —a $\sim 36\times$ uncertainty gap that forces hundreds of exploratory selections before the router can make informed decisions. With geometric forgetting ($\gamma=0.995$), arms selected infrequently see rising uncertainty as the prior decays. Frequently selected arms converge by step ~ 400 , consistent with the regret convergence in Figure 8.

split ($n=1,824$). To verify that this burn-in does not artificially inflate test-set performance, we ablate the amount of validation data used for online learning before test evaluation across three budget regimes: unconstrained, tight ($B=\$3.0\times 10^{-4}/\text{req}$), and moderate ($B=\$6.6\times 10^{-4}/\text{req}$).

Unconstrained regime. Five warmup conditions share identical priors (from train) and the same hyperparameters ($\alpha=0.1$, $n_{\text{eff}}=1,000$, $\gamma=0.995$); only the fraction of val used for burn-in varies (0%, 25%, 50%, 75%, 100%). Two tabula-rasa baselines (0% and 100% burn-in, $n_{\text{eff}}=1$) complete a 2×2 factorial design crossing prior type with burn-in level. All conditions are evaluated on the same held-out test split across 20 random seeds with paired Wilcoxon signed-rank tests against the 100% burn-in reference.

Key findings (unconstrained). No warmup burn-in level differs significantly from the 100% baseline on test regret (Table 9). The 0% condition—warmup priors with *no* online learning before test—achieves 74.0 cumulative regret, nominally *lower* than the 100% condition (74.4; $\Delta=-0.6\%$, $p=0.64$). Early regret (Regret@200) is lowest at 0% burn-in (5.9 vs. 7.7) because the priors are fully intact at the start of the test trajectory.

Forgetting–burn-in interaction. With $\gamma=0.995$, effective memory is ~ 200 steps. After 1,785 val steps, warmup priors are decayed by $\gamma^{1785} \approx 1.3\times 10^{-5}$ —effectively erased. This explains why the warmup and tabula-rasa conditions converge after 100% burn-in (74.4 vs. 75.4; $p=0.29$): both operate on the same ~ 200 recent online observations, and the original priors contribute nothing.

Prior value: a deployment latency optimisation. The only statistically significant gap in the unconstrained regime is between tabula rasa with *no* burn-in (81.8) and all warmup conditions (74.0–76.0; $p < 10^{-5}$), a +10.0% regret penalty concentrated in the first ~ 200 steps (R@200: 10.1 vs. 5.9). Once sufficient online evidence accumulates—either through val burn-in or directly on the test stream—tabula rasa converges to the same steady-state (tabula rasa 100% burn-in: 75.4, $p=0.29$ vs. warmup 100%).

Budget-constrained regimes (Table 10). To verify that these findings generalise to the budget-paced conditions that are the paper’s main contribution, we repeat the 2×2 factorial (priors $\times$ burn-in at 0%/100%) under the Primal-Dual BudgetPacer at tight and moderate targets. To isolate the effect of reward-model learning from pacer calibration, the BudgetPacer is *pre-calibrated* on the full validation set: all prompts are routed under the initial policy (priors or identity) and the resulting costs are fed to the pacer, but the bandit is *not* updated. All burn-in fractions therefore start the trial with an identically calibrated λ_t ; any remaining difference is attributable solely to reward learning.³ The pattern is consistent across all regimes:

- **Burn-in does not inflate.** Within each budget regime, Warmup 0% vs. Warmup 100% is non-significant (tight: 175.2 vs. 172.1, $p=0.40$; moderate: 150.8 vs. 149.4, $p=0.74$).
- **Priors matter at cold start (tight budgets).** Tabula Rasa with no burn-in has significantly higher regret at tight budgets (183.6 vs. 172.1, $\Delta=+6.7\%$, $p=0.002$), which survives Bonferroni correction across the three within-regime tests ($p_{\text{adj}}=0.006$). Prior to pacer pre-calibration this gap was larger ($\Delta=+6.0\%$, $p < 10^{-3}$), confirming that the original confound between cold-pacer and cold-bandit inflated the apparent harm of missing priors.
- **Convergence after burn-in.** With 100% burn-in, warmup and tabula rasa converge at every budget level (tight: 172.1 vs. 171.2, $p=0.78$; moderate: 149.4 vs. 149.5, $p=0.98$), confirming that geometric forgetting erases the prior regardless of budget pressure.
- **Moderate budgets reduce but do not eliminate the gap.** At moderate budgets, tabula rasa without burn-in shows a smaller penalty (155.6 vs. 149.4, $\Delta=+4.1\%$, $p=0.017$), which does not survive Bonferroni correction ($p_{\text{adj}}=0.051$). The budget constraint narrows the feasible arm set, reducing—but not fully eliminating—the harm of uninformed exploration.
- **Budget compliance is consistent.** Cost/target ratios range from 97.0% to 100.0% across all conditions, confirming that the BudgetPacer achieves comparable spending regardless of burn-in level or prior initialisation.

Implication. Warmup priors are best understood as a *deployment latency optimisation*: they eliminate the cold-start transient where uninformed exploration dominates, halving early regret (R@200: 5.9 vs. 10.1 unconstrained; 20.5 vs. 26.9 at tight budget). Their contribution is transient by design—geometric forgetting replaces off-line estimates with online evidence within ~ 200 effective-memory steps—so the router’s steady-state quality is determined by the online learner and budget pacer, not the initialisation. This holds *regardless of budget regime*: the BudgetPacer’s adaptive λ_t provides

³The oracle in this ablation is the quality-only oracle $\max_a r_a$ (i.e., it ignores cost). This inflates absolute regret relative to the cost-adjusted oracle used in the main paper but does not affect relative comparisons between conditions, which is the ablation’s purpose.

Table 9: Validation burn-in ablation—unconstrained regime (20 seeds, held-out test split). Δ is relative to the 100% burn-in reference. No warmup condition differs significantly from the reference; the only significant gap is tabula rasa without burn-in.

Priors	Burn-in	Regret	R@200	Δ	p
Warmup	0%	74.0 ± 0.5	5.9	-0.6%	0.64
Warmup	25%	76.0 ± 0.7	8.6	+2.2%	0.17
Warmup	50%	75.1 ± 0.8	7.9	+0.9%	0.56
Warmup	75%	75.1 ± 0.7	7.9	+1.0%	0.56
Warmup	100%	74.4 ± 0.8	7.7	—	—
Tabula Rasa	0%	81.8 ± 0.8	10.1	+10.0%	$< 10^{-5}$
Tabula Rasa	100%	75.4 ± 0.8	8.1	+1.4%	0.29

Table 10: Validation burn-in ablation—budget-constrained regimes (20 seeds, held-out test split, BudgetPacer active, pacer pre-calibrated). Δ is relative to the Warmup 100% reference within each regime. Cost/T is the mean cost per request as a fraction of the budget target (1.00 = at target). The burn-in finding holds under budget pressure: within each regime, Warmup 0% is non-significant vs. Warmup 100%, and cost compliance is consistent across conditions.

Budget	Priors	Burn-in	Regret	R@200	Cost/T	Δ
Tight	Warmup	0%	175.2 ± 2.9	20.5	1.00	+1.8%
	Warmup	100%	172.1 ± 1.9	18.5	1.00	—
	Tabula Rasa	0%	183.6 ± 2.6	26.9	0.99	+6.7%
	Tabula Rasa	100%	171.2 ± 2.3	18.7	1.00	-0.5%
Moderate	Warmup	0%	150.8 ± 3.1	18.3	0.99	+0.9%
	Warmup	100%	149.4 ± 1.9	16.5	0.99	—
	Tabula Rasa	0%	155.6 ± 2.1	20.0	0.97	+4.1%
	Tabula Rasa	100%	149.5 ± 2.2	15.6	0.99	+0.1%

the same closed-loop cost control whether the router starts from informed priors or from scratch. The case for warmup priors thus scales with the deployment’s sensitivity to early-stage regret: applications routing thousands of requests in the first minutes benefit substantially; systems that can tolerate a brief learning phase may forgo priors entirely and rely on the online learner alone.

A.4 Prior Mismatch Sensitivity (Figures 12–13)

The warmup ablation (Section A.2) demonstrated that well-calibrated priors reduce regret by 11% under stationary conditions. All experiments in this paper use priors trained on a representative training set drawn from the same distribution as the test traffic. In practice, however, priors may be trained on stale data, a different prompt mix, or an outdated model portfolio. This experiment stress-tests warmup priors under progressively worse calibration to quantify the cost of bad priors and identify when tabula rasa is the safer choice.

Design. We construct five prior-quality levels forming a mismatch gradient:

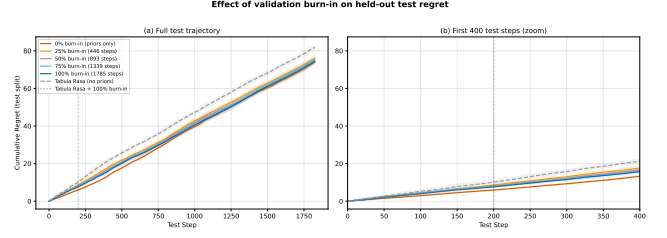


Figure 10: Validation burn-in ablation—unconstrained (K=3 stationary, 20 seeds, 95% bootstrap CI, held-out test split). Left: cumulative regret on test under varying burn-in fractions (0–100% of val). All warmup conditions are statistically indistinguishable ($p > 0.15$); the only significant gap is tabula rasa without burn-in (+10.0%, $p < 10^{-5}$). Right: zoom on the first 400 test steps—0% burn-in (priors only) has the lowest early regret because priors are fully intact.

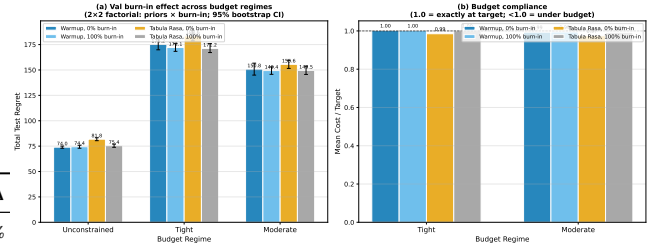


Figure 11: Validation burn-in ablation across budget regimes (20 seeds, 95% bootstrap CI; pacer pre-calibrated, see Appendix A.3). Left: total test regret. Within each regime, Warmup 0% and Warmup 100% are statistically indistinguishable, confirming that the burn-in finding holds under the BudgetPacer. The cold-start penalty (Tabula Rasa 0%) is significant at tight budgets (+6.7%, $p=0.002$, survives Bonferroni correction) and present at moderate budgets (+4.1%, $p=0.017$, non-significant after Bonferroni). Right: budget compliance (mean cost / target). All conditions stay within 3% of the budget target, confirming consistent pacer behaviour regardless of initialisation.

- (1) **Well-calibrated:** priors from the full training set ($n=8,374$)—the shipped default.
- (2) **Random-1680:** a random subsample of 1,680 training prompts, matching the GSM8K-only count while preserving the full distribution. This is a *sample-size control*: any gap between Random-1680 and Well-calibrated isolates covariance estimation noise from domain effects.
- (3) **MMLU-only:** priors from the 1,855 MMLU prompts only. The prior preserves the correct model ranking (Gemini > Mistral >> Llama) but with knowledge-domain-specific magnitudes.
- (4) **GSM8K-only:** priors from the 1,680 GSM8K (math) prompts only. All models score $\sim 0.95+$ on math, so the prior encodes near-zero arm differentiation.

(5) **Inverted:** priors from the full training set with Llama and Gemini rewards swapped, so the prior believes the cheapest model is best and vice versa.

Each level is tested at three n_{eff} values (10, 100, 1000) plus a single Tabula Rasa control ($n_{eff}=1$, no priors), yielding $5 \times 3 + 1 = 16$ conditions. All conditions share the Experiment 04 hyperparameters ($\alpha=0.1$, $\gamma=0.995$, disjoint LinUCB), which were tuned for the well-calibrated case and *not* re-optimised per condition—matching the production scenario where prior quality degrades after deployment without hyperparameter adjustment. Evaluation follows the same cumulative-regret protocol as the warmup ablation: held-out test split ($n=1,824$), 20 seeds (offset=9,000, aligned for paired comparisons), unconstrained (no budget pacer).

Results. Figure 12 summarises total regret across the quality-strength grid; pairwise comparisons use two-sided paired t -tests (19 d.f.) against the Tabula Rasa baseline (total regret 81.7 ± 4.2).

Directionally correct priors help at every strength. Well-calibrated priors reduce total regret monotonically with n_{eff} : -4.5% at $n_{eff}=10$ ($p < 0.001$), -4.6% at 100 ($p=0.004$), and -8.8% at 1,000 ($p < 10^{-4}$). The sample-size control (Random-1680) performs comparably to Well-calibrated at every n_{eff} —confirming that the n_{eff} scaling normalises prior strength and the benefit is not an artifact of having more training prompts.

Domain-mismatched priors still beat cold start. Both MMLU-only and GSM8K-only priors reduce regret relative to Tabula Rasa at $n_{eff} \geq 100$ ($p < 0.05$), despite being trained on single-domain data. At $n_{eff}=10$ the improvements are directionally positive but not statistically significant ($p > 0.09$), reflecting the weak prior’s limited initial influence. Crucially, no domain-mismatched prior *increases* regret at any n_{eff} : partial knowledge is always at least as good as no knowledge.

Inverted priors cause harm that scales with n_{eff} . This is the only condition where priors hurt. At $n_{eff}=10$, the damage is modest and non-significant ($+2.9\%$, $p=0.055$); at $n_{eff}=100$ it is significant ($+4.1\%$, $p=0.003$); and at $n_{eff}=1,000$ regret rises to 92.5 ($+13.3\%$, $p < 10^{-6}$), substantially worse than cold start. The mechanism is clear: the inverted prior confidently routes traffic to the worst model, and high n_{eff} forces the router to accumulate many online observations before overriding the prior.

Practical implications. The results establish that the prior-quality threshold for harm is high: priors must be *actively wrong* (systematically inverting model rankings) before they degrade performance. Domain mismatch, reduced sample size, and even near-zero arm differentiation (GSM8K-only) do not cross this threshold. For practitioners:

- When the prior is trained on representative data from the deployment domain, use $n_{eff}=1,000$ —this captures the full 8.8% regret reduction.
- When the prior comes from a related but different domain (e.g., a general-purpose prior applied to a specialised deployment), $n_{eff}=100$ is a safe default that still reduces regret by 3–4%.
- When prior quality is entirely unknown, set $n_{eff}=10$. This limits worst-case inverted-prior damage to $< 3\%$ (non-significant) while still providing a directional nudge from any non-adversarial prior.

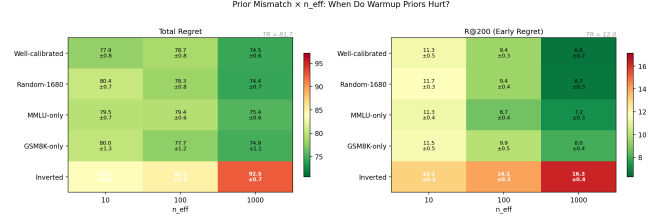


Figure 12: Total regret across prior quality levels and n_{eff} values ($K=3$ stationary, 20 seeds, held-out test split). The dashed line marks the Tabula Rasa baseline (81.7). Directionally correct priors (Well-calibrated through GSM8K-only) reduce regret at every n_{eff} ; only the Inverted prior—where model rankings are systematically swapped—causes harm, and the damage scales with n_{eff} .

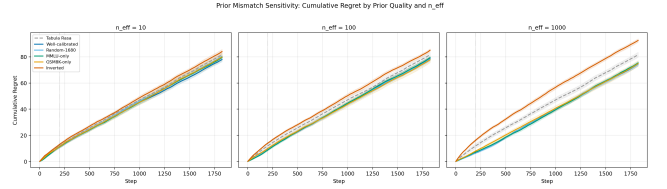


Figure 13: Cumulative regret curves for selected prior-quality conditions at $n_{eff}=1,000$ ($K=3$ stationary, 20 seeds, 95% bootstrap CI). Well-calibrated and domain-mismatched priors separate from Tabula Rasa within the first 200 steps and maintain the advantage throughout. The Inverted prior accumulates excess regret early as the router confidently routes to the wrong model, requiring hundreds of steps to partially recover.

- Tabula rasa ($n_{eff}=1$) is the appropriate choice only when the prior is known or suspected to be systematically inverted—a scenario that implies a fundamental error in the offline reward pipeline.

A.5 Model Onboarding Under Budget Constraints

Production LLM portfolios are not static. Providers release new models monthly—Gemini 2.5 Flash launched five months after Gemini 2.5 Pro, GPT-4o-mini arrived three months after GPT-4o—and platform teams need to incorporate them without disrupting cost compliance or quality. This section evaluates whether a single API call, `register_model()`, suffices to onboard a new model into a *budget-constrained* router.

A.5.1 Why Care? The standard industry workflow for integrating a new model into a supervised router is:

- (1) Collect the new model’s responses on a representative prompt set and label them (days to weeks of human or LLM-judge annotation).
- (2) Retrain the routing policy on the expanded labeled dataset.
- (3) Update hard-coded cost tables and budget thresholds.
- (4) Re-deploy and monitor for regressions.

This process is slow, manual, and error-prone: stale cost tables are the leading cause of budget violations in production routers (internal post-mortems). Critically, step 1 requires a *pre-collected evaluation dataset* for the new model before it can receive any live traffic—an offline data-collection campaign that must be repeated for every model addition.

ParetoBandit replaces steps 1–4 with a single call that seeds the new arm with a T-shirt prior and lets the online learner discover its quality–cost profile from live traffic. The bandit still requires a reward signal, but that signal comes from the same evaluation mechanism (LLM judge, user ratings, or implicit behavioural composites) already serving the existing portfolio—no dedicated offline dataset is collected for the new model. When the exploration mechanism routes a prompt to the new arm, the existing reward pipeline evaluates its response at zero marginal data-collection cost, and the bandit updates its posterior immediately. This also avoids two risks inherent to offline evaluation: *distribution mismatch* (a curated eval set may not reflect the live prompt distribution) and *dataset staleness* (the eval ages as deployment patterns drift). The BudgetPacer automatically accommodates the new arm’s cost without any reconfiguration.

A.5.2 Experimental Design. The same $K=3$ portfolio, data splits, and warmup priors used in Experiments 01–03 are deployed with the BudgetPacer. After Phase 1 online learning on the validation split ($n=1,785$), `register_model()` adds Gemini-2.5-Flash as a fourth arm with `speed="fast"` and its published pricing (input: \$0.30/M, output: \$2.50/M, blended: \$1.40/M). Phase 2 continues online on the $K=4$ test split ($n=1,824$), where Flash rewards are judged by DeepSeek-R1 using the same v3 continuous rubric as all other arms.

All conditions use the simulation-tuned hyperparameters from Experiments 02–03: $\alpha=0.1$, $n_{\text{eff}}=10$, $\gamma=0.995$, consistent across all main-body experiments.

Two Phase 2 strategies are compared per budget tier:

- **Fixed Policy (uniform 1/4):** Equal allocation across all $K=4$ arms. The simplest possible onboarding strategy—no routing intelligence, no budget awareness.
- **ParetoBandit (transfer):** Phase 1 posteriors carry over; Flash gets a T-shirt prior. BudgetPacer continues without reconfiguration.

Three budget targets span the constraint regime: tight ($B=\$3.0\times 10^{-4}/\text{req}$), moderate ($B=\$6.6\times 10^{-4}/\text{req}$), and loose ($B=\$1.9\times 10^{-3}/\text{req}$). An unconstrained ParetoBandit baseline (no pacer) serves as a quality ceiling. Results: 20 seeds (9000–9019).

A.5.3 Results (Figure 14).

Strategy	Budget	Reward	Cost (\$/req)	Flash %	Compliance
Fixed Policy	any	0.893	\$4.10e-3	25.1%	No (2–14X)
ParetoBandit	tight	0.857	\$3.00e-4	13.8%	✓
ParetoBandit	moderate	0.886	\$6.60e-4	22.6%	✓
ParetoBandit	loose	0.911	\$1.82e-3	30.6%	✓
ParetoBandit	unconstrained	0.923	\$7.59e-3	21.0%	—

Flash is discovered in every seed. Across all 80 ParetoBandit trials (20 seeds $\times$ 4 budget tiers), Flash is first selected within ~ 210 post-onboarding steps and reaches sustained adoption (windowed share $> 10\%$ for 50 consecutive steps) by step ~ 233 . Under the unconstrained ceiling, Flash stabilises at $21.0 \pm 1.0\%$ of traffic—comparable to its uniform-1/4 share but *contextually allocated* (the bandit routes Flash to prompts where it matches Gemini-Pro quality at $10\times$ lower cost).

Budget determines adoption speed, not whether adoption occurs. At loose budgets, Flash reaches 38% of traffic in the final 200 steps, exceeding the unconstrained ceiling (25%). At tight budgets, the pacer limits Flash to 14%, just above the sustained-adoption threshold—the router correctly identifies that Flash (blended \$1.40/M) is cheaper than Mistral (\$1.00/M) but more expensive than Llama (\$0.10/M), and allocates accordingly.

Flash displaces the right arm. Displacement analysis (early 200 vs. late 200 steps of Phase 2) reveals a clear pattern:

- **Under budget pressure** (tight and moderate): Flash takes share primarily from **Mistral-Large** (-27.3 pp at tight, -22.9 pp at moderate). This is economically rational: Flash offers Mistral-level quality (~ 0.92) at a lower blended cost (\$1.40 vs. \$1.00/M).
- **Unconstrained:** Flash takes share from **Gemini-Pro** (-20.8 pp). Without cost pressure, the bandit uses Flash as a near-Pro-quality alternative at $10\times$ lower cost.

The BudgetPacer requires no reconfiguration. Figure 14(c) shows cost compliance throughout the trajectory. ParetoBandit meets each budget target to within 1% across both phases, despite the portfolio expanding from $K=3$ to $K=4$ mid-trajectory. The dual variable λ_t adjusts smoothly: when Flash’s lower cost reduces the running average spend, λ_t relaxes, freeing budget for contextual exploration.

Fixed Policy: budget-oblivious baseline. The uniform-1/4 baseline highlights the cost of ignoring budget constraints. By giving Gemini-Pro 25% of traffic, it spends $\$4.10e-3$ per request—exceeding the tight budget by $14\times$ and even the loose budget by $2\times$. It achieves a reward of 0.893, which ParetoBandit nearly matches at moderate budgets (0.886) and exceeds at loose budgets (0.911) while spending $6\times$ and $2\times$ less.

A.5.4 Practical Implications.

One API call, zero downtime. At moderate and loose budgets—the regime of most production deployments—onboarding a new model into ParetoBandit requires a single call to `register_model()` with the model’s pricing. No offline evaluation, no policy retraining, no budget reconfiguration. The bandit discovers the model’s quality–cost niche from live traffic and the pacer auto-adjusts.

Budget-quality Pareto improvement. Flash is a Pareto-improving arm: it offers near-Pro quality at Mistral-level cost. A static router would require human analysis to discover this; ParetoBandit identifies and exploits it automatically, shifting the portfolio’s cost–quality frontier.

Tight budgets require deliberate exploration. Under tight budget constraints, the pacer’s exploration budget is limited, slowing (but not preventing) Flash adoption. Practitioners running at very tight margins who want to onboard an expensive newcomer should

temporarily increase the budget target to allow initial exploration, then tighten once the bandit calibrates.

A.5.5 Uncertainty Evolution: Warm-Start vs. Cold-Start (Figure 15). To visualise the mechanism driving Flash’s adoption, Figure 15 plots $\text{tr}(A_a^{-1})$ —a scalar summary of each arm’s total uncertainty—across both phases. Lower values indicate more accumulated information and tighter confidence bounds; higher values indicate greater uncertainty and larger UCB exploration bonuses.

At initialisation, the three warm-started arms begin with $\text{tr}(A_a^{-1}) \approx 0.6$, reflecting the $n_{\text{eff}}=1,000$ pseudo-observations encoded in their warmup priors. When Flash is registered at the Phase 2 boundary, it starts at $\text{tr}(A_a^{-1}) = d/\lambda = 26$ —the isotropic prior $A = \lambda I$ with no prior information. This $\sim 40\times$ uncertainty gap between Flash and the incumbents is the direct cause of LinUCB’s exploration burst: the UCB bonus $\alpha\sqrt{x^\top A_a^{-1} x}$ is proportional to the arm’s uncertainty, so Flash receives a massive selection advantage until observations shrink its confidence ellipsoid. Within ~ 300 Phase 2 steps, Flash’s uncertainty converges to the level of the warm-started arms, at which point its selection rate stabilises and reflects its learned quality–cost profile rather than exploration pressure.

The figure also reveals an instructive side effect for Gemini-Pro, the most expensive arm. Under budget constraints, Gemini-Pro is rarely selected ($< 2\%$ of Phase 2 traffic at the moderate and tight targets). Because geometric forgetting continuously decays A_a with each step the arm goes unplayed, Gemini-Pro’s uncertainty *rises* over time—the router progressively “forgets” what it knows about an arm it can no longer afford to play. The sawtooth pattern reflects the interplay: uncertainty accumulates during long idle periods, occasionally triggering an exploratory selection when the inflated UCB bonus overcomes the budget penalty, which briefly reduces uncertainty before the cycle repeats. This is a concrete illustration of the forgetting–budget pacer feedback loop discussed in the design rationale (Section 4): forgetting inflates confidence, which drives exploration, which incurs cost, which drives up λ_t , which suppresses the expensive arm again.

A.6 Cost Heuristic Validation (Figure 16)

The selection utility (Equation 3) uses a static log-normalized cost $\tilde{c}_a$ derived from each model’s blended per-token rate rather than the realised per-request cost. We validate this design choice by testing two necessary conditions: (i) $\tilde{c}_a$ preserves the *true cost ranking* across prompts, and (ii) within-model cost variance is small relative to inter-model gaps in $\tilde{c}$ -space.

K=3 portfolio (1,785 val prompts). The heuristic ordering Llama-8B $<$ Mistral-Large $<$ Gemini-Pro matches the per-request cost ordering on 99.9% of prompts. All three pairwise comparisons hold at $\geq 99.9\%$. In log-normalised $\tilde{c}$ -space, the within-model standard deviation is 3–17% of the total inter-model gap, confirming that prompt-level cost noise does not overwhelm the static heuristic signal.

K=4 portfolio (1,785 val prompts, with Gemini-Flash). Adding a fourth arm with similar pricing to an existing arm is a harder test. Gemini-Flash ($\tilde{c} = 0.441$) is placed between Mistral-Large ($\tilde{c} = 0.384$) and Gemini-Pro ($\tilde{c} = 0.673$). The full ordering matches

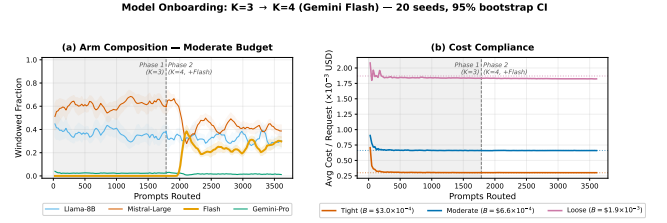


Figure 14:

Model onboarding under budget constraints ($K=3 \rightarrow K=4$). A $K=3$ router (Llama-8B, Mistral-Large, Gemini-Pro) learns on 1,785 prompts with warmup priors, then onboards Gemini-2.5-Flash as a fourth arm via `register_model()` (dashed vertical line). Phase 2 continues online on the $K=4$ test split (1,824 prompts). (a) Flash adoption trajectory: the windowed selection fraction of Flash rises from zero after onboarding and stabilises within ~ 230 steps across all budget tiers. Loose budgets reach $\sim 38\%$ Flash share; tight budgets plateau at $\sim 14\%$. The grey dotted line marks 25% (uniform $1/4$ baseline). (b) Full arm composition under the moderate budget: Flash primarily displaces Mistral-Large (-22.9 pp), taking over the mid-tier quality niche at lower cost. (c) Cost compliance on log scale: ParetoBandit (solid) tracks each budget target (dotted) within $\sim 2\%$ across the full trajectory, while the Fixed Policy (dashed grey) exceeds all targets by 2–14 $\times$. 95% bootstrap CI across 20 seeds.

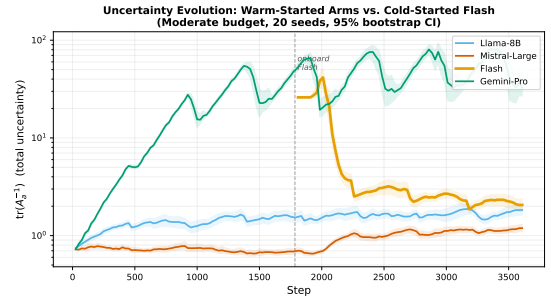


Figure 15: Uncertainty evolution during model onboarding (moderate budget, 20 seeds, 95% bootstrap CI). $\text{tr}(A_a^{-1})$ measures total uncertainty per arm: higher values indicate less information and larger exploration bonuses. Warm-started arms (Llama-8B, Mistral-Large) maintain low, stable uncertainty (~ 0.6 – 2) throughout both phases. Cold-started Flash appears at the Phase 2 boundary with $\text{tr}(A_a^{-1}) = d/\lambda = 26$ (the isotropic prior $A = \lambda I$) and converges to the incumbents’ level within ~ 300 steps as LinUCB’s exploration bonus drives traffic to it. Gemini-Pro exhibits rising, oscillatory uncertainty because it is rarely selected under budget pressure—geometric forgetting erases its accumulated information faster than new observations replenish it, illustrating the forgetting–budget pacer feedback loop described in Section 4.

on 79.2% of prompts; the closest pair (Mistral-Large vs. Gemini-Flash) preserves ranking on only 79.3% of prompts, reflecting Flash’s

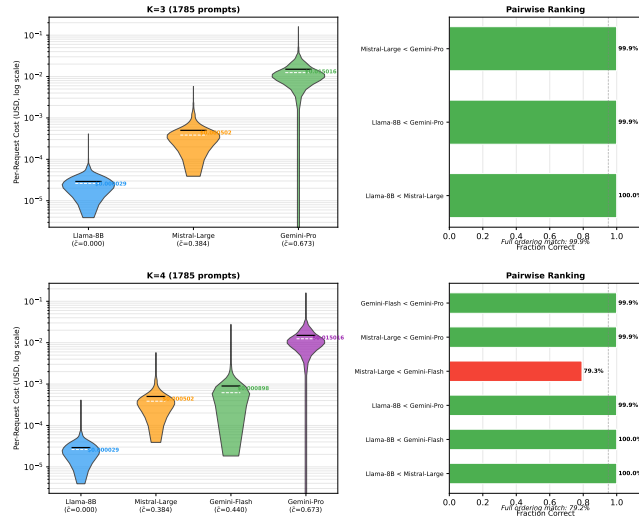


Figure 16: Cost heuristic validation. Left: violin plots of actual per-request costs (log scale) for each model, ordered by the static heuristic $\tilde{c}_a$. Right: fraction of prompts where each pairwise cost ordering matches the heuristic ranking. Top row: $K=3$ portfolio; bottom row: $K=4$ with Gemini-Flash added.

high cost variance ($CV = 1.55$) and the narrow inter-model gap ($\Delta\tilde{c} = 0.056$).

Prompt-cost correlation. Spearman correlations between prompt word count and per-request cost are weak ($\rho = 0.10$ – 0.26), indicating that prompt features are poor predictors of model cost in this portfolio. A contextual cost model would add complexity with minimal benefit.

Cross-model cost correlation. Per-request costs are moderately to strongly correlated across models ($\rho = 0.56$ – 0.72), driven by shared output-length variation. This shared factor preserves relative ordering even when absolute costs vary widely ($CV = 0.62$ – 1.55).

Takeaway. The static $\tilde{c}_a$ heuristic is empirically well-calibrated for portfolios where models span at least one order of magnitude in pricing. For denser price clusters (e.g., Mistral-Large vs. Gemini-Flash, where the heuristic gap is only $\Delta\tilde{c} = 0.056$), the ranking holds for $\sim 79\%$ of prompts but inverts when Flash generates short responses. The budget pacer’s dual variable λ_t , which updates on *actual* costs, compensates for these per-request deviations from the static estimate.